

目录

第一章 概述	5
1.1 强化学习能解决的问题	5
1.2 强化学习与其他机器学习方法的联系和区别	5
1.2.1 与监督学习的联系和区别	5
1.2.2 与传统优化和最优控制的联系和区别	6
1.3 强化学习如何解决问题	6
1.4 强化学习的分类	7
1.4.1 按是否依赖环境模型分类	7
1.4.2 按策略更新方法分类	7
1.4.3 按回报函数是否已知分类	7
1.4.4 按任务结构和规模分类	7
第二章 有限马尔科夫决策过程	8
2.1 从多臂赌博机到马尔科夫决策过程	8
2.1.1 多臂赌博机的例子计算	8
2.2 多臂赌博机与 MDP 的区别	9
2.3 马尔科夫性	10
2.4 有限马尔科夫决策过程的定义	10
2.5 状态、动作与回报的设计	10
2.6 累积回报与折扣回报	11
2.7 策略与值函数	11
2.8 贝尔曼方程	12
2.9 值函数的具体形式理解	12
2.9.1 从给定策略的值函数到最优值函数	13
2.10 最优价值函数、最优状态-动作值函数与最优策略	13
第三章 动态规划方法	15
3.1 动态规划的本质	15
3.2 动态规划与强化学习的关系	15
3.3 策略评估	15
3.3.1 解析法	16
3.3.2 迭代策略评估	16
3.4 策略改进	16
3.4.1 策略改进定理证明	17

目录	2
3.5 策略迭代	17
3.6 值函数迭代	18
3.7 动态规划方法小结	18
第四章 蒙特卡洛强化学习方法	19
4.1 模型未知与蒙特卡洛方法	19
4.2 蒙特卡洛策略评估	19
4.2.1 首次访问 MC 与每次访问 MC	20
4.3 蒙特卡洛评估行为值函数	20
4.4 MC 与 DP 策略评估的区别	20
4.5 蒙特卡洛策略改进	21
4.6 探索型初始化	21
4.7 无探索型初始化与永久温和策略	21
4.8 On-policy MC	22
4.9 Off-policy MC	22
4.10 重要性采样	22
4.10.1 一般重要性采样	22
4.10.2 MC 中的重要性权重	23
4.10.3 状态访问时刻集合	23
4.10.4 普通重要性采样估计	24
4.10.5 加权重要性采样估计	24
4.11 Off-policy Every-visit MC 控制	24
4.12 蒙特卡洛方法小结	25
第五章 时间差分及 DQN	26
5.1 时间差分学习的基本思想	26
5.2 TD 的偏差与方差	26
5.3 TD 学习的优势	27
5.4 Sarsa: On-policy TD 控制	27
5.4.1 Sarsa 课件伪代码	27
5.5 Q-learning: Off-policy TD 控制	28
5.5.1 Q-learning 课件伪代码	28
5.6 Expected Sarsa	29
5.7 Double Q-learning	29
第六章 函数逼近及 DQN	30
6.1 为什么需要函数逼近	30
6.2 函数逼近方法分类	30
6.3 值函数逼近的目标	31
6.4 半梯度方法	32
6.4.1 半梯度 TD(0) 值函数评估流程	32
6.4.2 半梯度 Sarsa	33
6.5 线性值函数逼近	33
6.6 批方法	34

目录	3
6.7 表格型强化学习是一种特殊的函数逼近	34
6.8 固定稀疏表示	35
6.9 神经网络值函数逼近	35
6.9.1 前向神经网络的正向传播	35
6.9.2 卷积神经网络	36
6.10 DQN	36
6.10.1 DQN 课件伪代码	37
6.11 DQN 的常见变种	38
第七章 基于策略梯度的方法	39
7.1 为什么需要策略搜索	39
7.2 策略搜索方法分类	39
7.3 似然率策略梯度	40
7.4 常见策略表示	42
7.5 随机策略梯度的推导及解释	42
7.6 REINFORCE	42
7.7 基线与方差减小	43
7.8 策略梯度定理	44
第八章 TRPO 及 PPO	45
8.1 为什么普通策略梯度需要改进	45
8.2 性能差分与替代回报函数	45
8.2.1 性能差分恒等式	45
8.3 折扣状态分布与精确替代目标	46
8.4 策略改善理论及其证明	46
8.5 从精确替代目标到可采样目标	47
8.5.1 忽略状态分布变化	47
8.5.2 用重要性采样处理动作分布	47
8.6 替代目标与真实目标的一阶一致性	47
8.7 单调改进下界	47
8.8 TRPO: 带 KL 约束的策略优化	48
8.9 TRPO 的近似求解: 共轭梯度与线性搜索	48
8.10 PPO: 用裁剪目标近似信赖域	49
8.11 PPO 的完整目标与 GAE	50
8.12 TRPO 用到的信息论概念	50
8.13 TRPO 与 PPO 对比	51
第九章 确定性策略梯度方法	52
9.1 从随机策略梯度到确定性策略梯度	52
9.2 Actor-Critic 框架	52
9.3 确定性值函数	52
9.4 确定性策略梯度定理	53
9.5 确定性 Actor-Critic	53
9.6 DDPG	53

目录	4
9.7 TD3	54
9.8 D4PG	54
第十章 多智能体强化学习	55
10.1 多智能体强化学习的分类	55
10.2 多智能体强化学习的挑战	55
10.3 博弈论基本概念	55
10.4 Best Response 与纳什均衡	56
10.5 零和博弈与 Minimax	56
10.6 马尔科夫博弈	56
10.7 多智能体 Q-learning	57
10.8 典型多智能体算法	57
10.9 深度多智能体强化学习	58
10.10 多智能体强化学习小结	58

第一章 概述

1.1 强化学习能解决的问题

强化学习 (Reinforcement Learning, RL) 主要研究智能体如何通过与环境交互来学习决策策略。它适合处理具有序贯决策、长期收益、试错探索等特征的问题，尤其适用于模型未知、系统复杂或难以直接写出解析优化目标的场景。

- **控制问题**：例如非线性系统控制、倒立摆控制、机器人站立、行走和路径跟随等。此类问题通常需要智能体根据当前状态连续选择动作，使系统长期表现最优。
- **游戏与博弈问题**：例如 AlphaGo、星际争霸、多智能体对抗等。游戏环境具有明确的状态、动作和胜负回报，适合用强化学习学习复杂策略。
- **多智能体协作与竞争**：例如启元世界、星际争霸、多无人机对抗等。多个智能体之间可能存在合作、竞争或混合关系。
- **复杂工程系统优化**：例如可控核聚变中的等离子体位形控制。系统模型复杂、精确建模困难时，可以通过交互数据学习控制策略。
- **大模型对齐与人类反馈学习**：ChatGPT 的训练中使用了基于人类反馈的强化学习思想，先训练监督微调策略，再训练回报模型，最后用 PPO 等算法优化策略。

因此，强化学习关注的不是单步预测是否正确，而是智能体在连续决策过程中如何通过动作影响未来状态，并最大化长期累积回报。

1.2 强化学习与其他机器学习方法的联系和区别

强化学习和其他机器学习方法的共同点是都从数据中学习，并通过学习逐步改善性能。但强化学习的数据来源、学习目标和反馈形式具有明显特点。

1.2.1 与监督学习的联系和区别

- **联系**：二者都可以看作从数据中学习函数映射，并且都可以使用神经网络等函数逼近工具。
- **数据不同**：监督学习依赖带标签的数据，例如输入 x 和标签 y ；强化学习依赖智能体与环境交互产生的数据，例如状态、动作、奖赏和下一状态。
- **目标不同**：监督学习通常最小化预测误差；强化学习最大化长期累积回报。
- **反馈不同**：监督学习的反馈通常是即时且明确的标签；强化学习的反馈可能是延迟的，一个动作的好坏常常要经过多个时间步之后才能体现。

- **探索问题：**强化学习需要在“探索未知动作”和“利用已有较优策略”之间权衡，而监督学习通常不涉及主动探索环境。

1.2.2 与传统优化和最优控制的联系和区别

强化学习也可以看作求解序贯最优决策问题的一类方法。对于状态空间很小、模型和回报函数已知的问题，可以使用线性规划或动态规划；对于模型已知、回报函数解析的问题，可以采用最优控制方法；对于策略形式较简单的问题，也可以使用进化算法等优化方法。

当系统模型未知、环境非常复杂、回报函数难以解析给出时，强化学习通过采样和交互来学习最优策略。也就是说，强化学习更强调“在交互中学习”，而不是完全依赖已知模型进行解析求解。

1.3 强化学习如何解决问题

强化学习的基本框架包括智能体、环境、状态、动作、奖赏、策略和值函数。

- **智能体 (agent)：**做决策的主体。
- **环境 (environment)：**智能体交互的对象，会根据动作转移到新状态并给出奖赏。
- **状态 (state)：**描述当前环境情况的信息。
- **动作 (action)：**智能体在某个状态下可以采取的行为。
- **策略 (policy)：**从状态到动作的映射，通常记为 $\pi(a|s)$ ，表示在状态 s 下选择动作 a 的概率。
- **奖赏 (reward)：**环境在每个时间步给智能体的即时反馈。
- **值函数 (value function)：**从某个状态出发，按照某个策略行动所能获得的长期累积回报。

强化学习的学习目标是寻找一个最优策略，使长期累积回报最大。其核心过程可以概括为：

1. 智能体观察当前状态 s_t ；
2. 根据策略 $\pi(a|s)$ 选择动作 a_t ；
3. 环境转移到下一状态 s_{t+1} ，并给出奖赏 r_t ；
4. 智能体利用交互数据更新策略或值函数；
5. 重复上述过程，逐步改善行为。

强化学习的两个重要特征是**试错探索**和**延迟回报**。智能体必须尝试不同动作来发现更优策略，同时还要处理“当前动作影响未来收益”的问题。

以井字棋为例，可以为每个局面设置一个值函数，表示从当前状态最终获胜的概率。赢的状态值为 1，输的状态值为 0，其他状态可以初始化为 0.5。在采样对局过程中，根据后继状态修正当前状态的价值：

$$V(s_t) \leftarrow V(s_t) + \alpha [V(s_{t+1}) - V(s_t)].$$

这体现了强化学习的基本思想：通过交互经验不断更新对状态或动作价值的估计，进而改进决策。

1.4 强化学习的分类

1.4.1 按是否依赖环境模型分类

- **基于模型的强化学习 (model-based RL)**: 先用数据学习环境模型, 再基于模型规划或求解最优策略。
- **无模型强化学习 (model-free RL)**: 不显式学习环境模型, 而是直接从交互数据中学习值函数或策略。

1.4.2 按策略更新方法分类

- **基于值函数的方法**: 先学习最优值函数, 再由值函数导出最优策略, 例如后续课程中的动态规划、蒙特卡洛、时间差分 and DQN 等方法。
- **直接策略搜索方法**: 直接在策略空间中优化策略参数, 例如策略梯度方法。
- **Actor-Critic 方法**: 同时学习策略和值函数。Actor 负责改进策略, Critic 负责评价当前策略, 思想上类似策略迭代。

1.4.3 按回报函数是否已知分类

- **正向强化学习**: 回报函数已知, 目标是从回报出发学习最优策略。
- **逆向强化学习**: 根据专家示例反推出回报函数, 再利用回报函数学习策略。

1.4.4 按任务结构和规模分类

面对更复杂的任务, 还可以进一步发展出分层强化学习、元强化学习、多目标强化学习、多任务强化学习、迁移强化学习和多智能体强化学习等方向。

第二章 有限马尔科夫决策过程

2.1 从多臂赌博机到马尔科夫决策过程

多臂赌博机 (multi-armed bandit) 是强化学习中最简单的例子。它只有一个状态，智能体每次从 K 个动作中选择一个动作，每个动作对应一个未知的回报分布。目标是在有限次尝试中获得尽可能高的总回报。

对于动作 a ，其真实动作价值定义为：

$$q_*(a) = \mathbb{E}[R_t | A_t = a].$$

由于真实期望通常未知，只能通过采样估计：

$$Q(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot \Pi_{A_i=a}}{\sum_{i=1}^{t-1} \Pi_{A_i=a}}.$$

其中， $\Pi_{A_i=a}$ 表示第 i 次选择的动作是否为 a ，若 $A_i = a$ 则取 1，否则取 0。

多臂赌博机的核心问题是探索-利用平衡：

- **利用 (exploitation)**：选择当前估计价值最高的动作，例如贪婪策略

$$a = \arg \max_a Q(a).$$

- **探索 (exploration)**：尝试当前看来不是最优的动作，以获得更多信息，避免过早陷入局部最优。

动作价值可以用增量形式更新：

$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n).$$

这个公式的意义是：新估计值 = 旧估计值 + 步长 $\times$ 预测误差。后续强化学习中的很多更新公式都可以看作这种思想的推广。

2.1.1 多臂赌博机的例子计算

假设有一个三臂赌博机，动作集合为：

$$A = \{a_1, a_2, a_3\}.$$

智能体前 6 次交互得到如下数据：

i	1	2	3	4	5	6
A_i	a_1	a_2	a_1	a_3	a_1	a_2
R_i	1	0	2	4	3	2

根据动作价值估计公式：

$$Q(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot \Pi_{A_i=a}}{\sum_{i=1}^{t-1} \Pi_{A_i=a}},$$

可以分别计算三个动作的估计价值。

对于动作 a_1 ，它在第 1, 3, 5 次被选择，对应回报为 1, 2, 3，因此：

$$Q(a_1) = \frac{1 + 2 + 3}{3} = 2.$$

对于动作 a_2 ，它在第 2, 6 次被选择，对应回报为 0, 2，因此：

$$Q(a_2) = \frac{0 + 2}{2} = 1.$$

对于动作 a_3 ，它只在第 4 次被选择，对应回报为 4，因此：

$$Q(a_3) = \frac{4}{1} = 4.$$

如果此时采用贪婪策略，则下一次会选择：

$$a = \arg \max_a Q(a) = a_3.$$

这个例子说明：多臂赌博机通过不断试验不同动作，用采样平均回报估计每个动作的价值，再根据动作价值选择动作。

也可以用增量公式计算。例如动作 a_1 第一次被选择后：

$$Q_1(a_1) = 1.$$

第二次选择 a_1 时获得回报 2，更新为：

$$Q_2(a_1) = Q_1(a_1) + \frac{1}{2} (2 - Q_1(a_1)) = 1 + \frac{1}{2} (2 - 1) = 1.5.$$

第三次选择 a_1 时获得回报 3，更新为：

$$Q_3(a_1) = Q_2(a_1) + \frac{1}{3} (3 - Q_2(a_1)) = 1.5 + \frac{1}{3} (3 - 1.5) = 2.$$

最终结果与直接求平均值一致。

2.2 多臂赌博机与 MDP 的区别

多臂赌博机可以看作单状态决策问题，动作只影响当前回报；而马尔科夫决策过程（Markov Decision Process, MDP）是多状态序贯决策问题，动作不仅影响立即回报，还会影响下一状态。

- **多臂赌博机**：在状态 s 下采取动作 a ，得到回报 r ，但状态不发生本质变化。
- **MDP**：在状态 s_t 下采取动作 a_t ，得到回报 r_t ，同时环境转移到下一状态 s_{t+1} 。

因此，MDP 中评价一个动作不能只看立即回报，还要看它把智能体带到什么状态。也就是说，动作价值需要同时考虑立即回报和未来回报。

2.3 马尔科夫性

马尔科夫性是 MDP 的基本假设：系统的下一个状态只与当前状态有关，与更早的历史无关。形式化表示为：

$$P(S_{t+1}|S_t) = P(S_{t+1}|S_1, \dots, S_t).$$

这表示：当前状态已经包含了做决策所需的全部相关历史信息。一旦当前状态已知，过去的历史就可以被“丢弃”。如果状态设计得不好，遗漏了重要历史信息，就会破坏马尔科夫性，从而影响强化学习效果。

2.4 有限马尔科夫决策过程的定义

有限马尔科夫决策过程通常由五元组表示：

$$(S, A, P, R, \gamma).$$

其中：

- **S ：状态集合。**状态是所有可能有助于决策的信息，例如棋盘局面、机器人姿态、图像帧等。
- **A ：动作集合。**动作是智能体可以做出的决定，例如下棋落子、机器人关节力矩、电机电压等。
- **P ：状态转移概率。**描述在状态 s 下采取动作 a 后转移到下一状态 s' 的概率。
- **R ：回报函数。**描述智能体采取动作后获得的立即回报。
- **γ ：折扣因子。**用于平衡当前回报和未来回报，满足 $0 \leq \gamma \leq 1$ 。

MDP 的动力学可以用联合条件概率表示：

$$p(s', r|s, a) = \Pr(S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a).$$

由此可以得到状态转移概率：

$$p(s'|s, a) = \sum_r p(s', r|s, a),$$

以及回报的期望：

$$r(s, a) = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a] = \sum_{r \in R} r \sum_{s' \in S} p(s', r|s, a).$$

2.5 状态、动作与回报的设计

MDP 是一个抽象框架，很多问题都可以建模为 MDP，但建模质量会直接影响学习效果。

- **状态设计：**状态应尽量包含做决策所需的信息。DQN 可以用连续图像帧作为状态，AlphaGo 可以用棋局历史作为状态。状态表示会严重影响学习效果。
- **动作设计：**动作可以是宏观行为，也可以是底层控制量。例如机器人走路时，动作可以是关节力矩、关节角度或电机电压。

- **回报设计**：回报应使“最大化累积回报”等价于完成任务目标。例如机器人走路可以用前向速度作为正回报，迷宫逃离可以每走一步给 -1 以鼓励尽快到达终点，下棋可以设置赢 $+1$ 、输 -1 、平局 0 。

回报函数是强化学习问题中非常关键的设计。如果回报设计不合理，智能体可能学到与真实目标不一致的行为。

2.6 累积回报与折扣回报

智能体的目标是**最大化累积回报**。对于有终止状态的幕式任务（episodic tasks），回报可以定义为：

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T.$$

对于没有明确终止状态的连续任务（continuing tasks），通常使用折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

折扣因子 γ 的含义：

- $\gamma = 0$ ：智能体只关心立即回报，是短视的。
- γ 接近 1 ：智能体更重视未来回报，是有远见的。
- $0 < \gamma < 1$ ：在当前收益和未来收益之间折中，同时保证无限序列回报在一定条件下有界。

2.7 策略与值函数

策略（policy）描述智能体在状态 s 下如何选择动作。随机策略定义为：

$$\pi(a|s) = P(A_t = a | S_t = s).$$

常见策略包括：

- **贪婪策略**：总是选择当前价值最大的动作。
- **随机策略**：在动作集合上按照某种概率分布采样。
- **玻尔兹曼策略**：根据动作价值的指数形式分配动作概率，使价值越高的动作被选中的概率越大。

状态值函数表示在策略 π 下，从状态 s 出发能够获得的期望折扣累积回报：

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right].$$

动作值函数表示在策略 π 下，从状态 s 出发，先采取动作 a ，之后继续按照策略 π 行动所能获得的期望折扣累积回报：

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right].$$

值函数的核心作用是**评价状态或动作的长期好坏**，不是只评价当前一步的回报。

2.8 贝尔曼方程

贝尔曼方程体现了值函数的递推结构：当前状态的价值 = 当前立即回报 + 后继状态价值的折扣期望。

状态值函数与动作值函数之间有如下关系：

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) q_{\pi}(s, a).$$

动作值函数可以写成：

$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s').$$

将二者合并可得状态值函数的贝尔曼方程：

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s') \right].$$

动作值函数也可以递推为：

$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') q_{\pi}(s', a').$$

贝尔曼方程是后续动态规划、蒙特卡洛、时间差分 and Q-learning 等方法的理论基础。

2.9 值函数的具体形式理解

课件中给出了一个五状态的小例子，用来说明值函数如何由策略、立即回报、状态转移概率和后继状态价值共同决定。

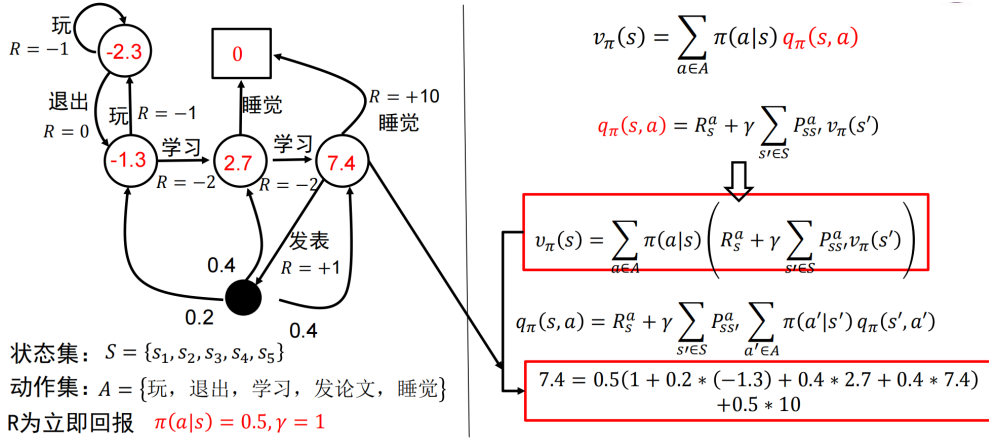


图 2.1: 值函数的具体形式理解示意图

设状态集合为：

$$S = \{s_1, s_2, s_3, s_4, s_5\},$$

动作集合为：

$$A = \{\text{玩, 退出, 学习, 发论文, 睡觉}\}.$$

假设折扣因子为：

$$\gamma = 1,$$

策略在某些状态下以概率 0.5 选择不同动作，即：

$$\pi(a|s) = 0.5.$$

值函数的计算依据是：

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) q_{\pi}(s, a),$$

其中动作值函数为：

$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s').$$

以课件中值为 7.4 的状态为例，该状态下有两个可能动作：**发论文**和**睡觉**，二者被选择的概率均为 0.5。

- 若选择**发论文**，立即回报为 +1，之后分别以概率 0.2, 0.4, 0.4 转移到价值为 -1.3, 2.7, 7.4 的状态。
- 若选择**睡觉**，立即回报为 +10，可看作直接得到该动作的回报。

因此该状态的值函数可计算为：

$$\begin{aligned} v_{\pi}(s) &= 0.5 [1 + 0.2 \times (-1.3) + 0.4 \times 2.7 + 0.4 \times 7.4] + 0.5 \times 10 \\ &= 0.5 [1 - 0.26 + 1.08 + 2.96] + 5 \\ &= 0.5 \times 4.78 + 5 \\ &= 7.39 \approx 7.4. \end{aligned}$$

这个例子说明：一个状态的价值不是单纯由当前回报决定，而是由当前策略下所有可能动作的期望价值共同决定；每个动作的价值又由立即回报和后继状态价值共同决定。

2.9.1 从给定策略的值函数到最优值函数

上面的计算是在**给定策略** π 的前提下进行的。例如在某个状态下，策略以 0.5 的概率选择“发论文”，以 0.5 的概率选择“睡觉”，于是状态价值是这两个动作价值的加权平均。因此，它得到的是：

$$v_{\pi}(s), \quad q_{\pi}(s, a),$$

也就是**当前策略**下的状态价值和动作价值。

但强化学习的最终目标不是只评价当前策略，而是要找到最优策略。因此接下来要从“当前策略有多好”推进到“所有策略中最好能有多好”。这就引出了**最优状态值函数**、**最优状态-动作值函数**和**最优策略**。

2.10 最优价值函数、最优状态-动作值函数与最优策略

最优状态值函数是在所有策略中状态价值最大的值函数：

$$v_*(s) = \max_{\pi} v_{\pi}(s).$$

最优动作值函数是在所有策略中动作价值最大的值函数：

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a).$$

可以将这几个符号对比理解：

- $v_\pi(s)$: 在给定策略 π 下, 从状态 s 出发的期望长期回报。
- $q_\pi(s, a)$: 在给定策略 π 下, 先在状态 s 执行动作 a , 之后继续按 π 行动的期望长期回报。
- $v_*(s)$: 在所有策略中选择最好的策略后, 状态 s 能达到的最大价值。
- $q_*(s, a)$: 先在状态 s 执行动作 a , 之后都按最优方式行动时能达到的最大价值。

因此, 前面例子中算出的 7.4 是**某个给定策略下的值**; 而最优价值函数关心的是: 如果每一步都尽可能选择最优动作, 该状态最多能获得多少长期回报。

对应的贝尔曼最优方程为:

$$v_*(s) = \max_a \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_*(s') \right],$$

$$q_*(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \max_{a'} q_*(s', a').$$

这两个公式与前面的贝尔曼期望方程相比, 核心变化是把“按照策略 π 加权平均”变成了“对动作取最大值”。也就是说:

- **贝尔曼期望方程**: 已经给定策略, 只计算该策略下的平均表现。
- **贝尔曼最优方程**: 不固定策略, 而是在每个状态选择使长期回报最大的动作。

如果已经知道最优动作值函数 $q_*(s, a)$, 那么最优策略可以直接由贪婪方式得到:

$$\pi_*(a|s) = \begin{cases} 1, & a = \arg \max_{a \in A} q_*(s, a), \\ 0, & \text{otherwise.} \end{cases}$$

也就是说, 一旦知道 $q_*(s, a)$, 最优策略就不需要再额外搜索: 每个状态直接选择 $q_*(s, a)$ 最大的动作即可。

因此, 有限 MDP 中强化学习的形式化目标可以理解为: 通过与环境交互学习值函数或策略, 使智能体在长期意义下获得最大累积回报。从逻辑上看, 第二章后半部分的内容是:

1. 先用具体例子理解给定策略下的 $v_\pi(s)$ 和 $q_\pi(s, a)$;
2. 再把“给定策略的价值”推广为“所有策略中最大的价值”;
3. 最后利用最优动作值函数 $q_*(s, a)$ 构造最优策略 π_* 。

第三章 动态规划方法

3.1 动态规划的本质

动态规划（Dynamic Programming, DP）的本质是利用贝尔曼方程，将多阶段决策问题分解为多个单阶段决策问题。如果已知环境模型，即状态转移概率和回报函数，就可以通过递推或迭代的方式求解值函数，再由值函数重构最优策略。

对于离散多阶段决策问题，最优代价函数可以写成：

$$J^*[x(j), j] = \min_{u(j) \in U} \{L[x(j), u(j), j] + J^*[x(j+1), j+1]\}.$$

若系统满足状态转移关系

$$x(j+1) = f[x(j), u(j), j],$$

则有：

$$J^*[x(j), j] = \min_{u(j) \in U} \{L[x(j), u(j), j] + J^*[f[x(j), u(j), j], j+1]\}.$$

这说明：只要知道当前动作带来的立即代价，以及下一阶段的最优代价，就可以递推出当前阶段的最优决策。

3.2 动态规划与强化学习的关系

动态规划和强化学习都围绕值函数、策略评估和策略改进展开。二者的主要区别在于：

- 动态规划：假设 MDP 的模型已知，即已知 $P_{ss'}^a$ 和 R_s^a ，可以进行精确的期望更新。
- 强化学习：通常模型未知，需要通过与环境交互采样，用数据近似动态规划中的更新过程。

因此，可以把很多强化学习算法理解为不依赖完美模型、计算量更小的近似动态规划方法。

3.3 策略评估

策略评估（policy evaluation）的目标是：在给定策略 π 的情况下，求出该策略对应的状态值函数 $v_\pi(s)$ 。

在策略 π 下，状态值函数定义为：

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right].$$

动作值函数定义为：

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right].$$

根据贝尔曼方程，给定策略 π 时有：

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s') \right].$$

由于模型已知，上式中唯一未知的是各状态的值函数，因此可以看成是一个线性方程组。状态数为 $|S|$ 时，未知数个数就是 $|S|$ 。

3.3.1 解析法

将所有状态的值函数写成向量形式，可得：

$$V_{\pi} = C_{\pi} + \gamma P_{\pi} V_{\pi}.$$

其中：

$$C_{\pi}(s) = \sum_a \pi(a|s) r(s, a),$$

$$P_{\pi}(s, s') = \sum_a \pi(a|s) p(s'|s, a).$$

因此：

$$V_{\pi} = (I - \gamma P_{\pi})^{-1} C_{\pi}.$$

解析法能够直接求出值函数，但当状态空间很大时，矩阵求逆的计算代价很高。

3.3.2 迭代策略评估

实际中更常用数值迭代。给定初始值函数 $v_0(s)$ ，重复进行期望更新：

$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right].$$

这个过程可以理解为对所有状态进行反复扫描，每次用旧值函数 v_k 更新新值函数 v_{k+1} ，直到值函数收敛：

$$v_{k+1} \approx v_k.$$

策略评估解决的是“给定策略有多好”的问题。

3.4 策略改进

策略改进 (policy improvement) 的目标是：利用当前策略的值函数，构造一个更好的策略。给定当前策略 π_l 及其动作值函数 $q_{\pi_l}(s, a)$ ，最直接的改进方法是在每个状态采取贪婪动作：

$$\pi_{l+1}(s) \in \arg \max_a q_{\pi_l}(s, a).$$

其中：

$$q_{\pi_l}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi_l}(s').$$

策略改进定理的核心思想是：如果新策略在每个状态下选择的动作不比旧策略差，即

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s),$$

那么新策略整体上不比旧策略差：

$$v_{\pi'}(s) \geq v_{\pi}(s).$$

3.4.1 策略改进定理证明

由动作值函数定义可知：

$$q_{\pi}(s, \pi'(s)) = \mathbb{E}_{\pi'} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s].$$

若对任意状态 s 都有

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s),$$

则：

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) \\ &= \mathbb{E}_{\pi'} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) | S_t = s] \\ &= \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 v_{\pi}(S_{t+2}) | S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots | S_t = s] \\ &= v_{\pi'}(s). \end{aligned}$$

因此，新策略 π' 的值函数不小于旧策略 π 的值函数，即：

$$v_{\pi'}(s) \geq v_{\pi}(s).$$

这说明只要新策略在每个状态下选择的动作相对于旧策略都是更优或不差的，那么整体策略就会得到改进。

因此，策略改进解决的是“如何根据当前值函数找到更好策略”的问题。

3.5 策略迭代

策略迭代（policy iteration）交替执行策略评估和策略改进：

$$\pi_0 \xrightarrow{\text{策略评估}} v_{\pi_0} \xrightarrow{\text{策略改进}} \pi_1 \xrightarrow{\text{策略评估}} v_{\pi_1} \xrightarrow{\text{策略改进}} \cdots \rightarrow \pi_*.$$

算法流程可以概括为：

1. 初始化策略 π_0 和值函数 $V(s)$ ；
2. 对当前策略 π_l 做策略评估，得到 V_{π_l} ；
3. 对 V_{π_l} 做贪婪策略改进，得到 π_{l+1} ；
4. 如果 $\pi_{l+1} = \pi_l$ ，说明策略稳定，停止；
5. 否则继续重复评估和改进。

策略迭代的关键思想是：先把当前策略评价清楚，再基于评价结果改进策略。

3.6 值函数迭代

策略迭代中，策略改进通常要等策略评估收敛后再进行。但实际上，**不一定要等值函数完全收敛才改进策略**。如果每次只进行少量评估，甚至只进行一次状态扫描，就立刻做贪婪改进，就得到值函数迭代。

值函数迭代直接使用贝尔曼最优方程更新：

$$v_{l+1}(s) = \max_a \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_l(s') \right].$$

当值函数收敛后，可以由最优值函数恢复最优策略：

$$\pi(s) = \arg \max_a \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v(s') \right].$$

值函数迭代把策略评估和策略改进合并在一个更新公式中。它不显式保存完整策略，而是不断逼近最优值函数。

3.7 动态规划方法小结

动态规划方法主要包括：

- **策略评估**：给定策略，计算其值函数。
- **策略改进**：根据值函数构造更优策略。
- **策略迭代**：反复执行策略评估和策略改进，直到策略稳定。
- **值函数迭代**：直接使用贝尔曼最优方程逼近最优值函数。

动态规划的优点是理论清晰、更新目标明确；缺点是**要求环境模型已知，并且在大规模状态空间下计算量很大**。这也是后续蒙特卡洛方法、时间差分方法和深度强化学习要解决的重要问题。

第四章 蒙特卡洛强化学习方法

4.1 模型未知与蒙特卡洛方法

动态规划方法要求环境模型已知，即需要知道状态转移概率 $P_{ss'}^a$ 和回报函数 R_s^a 。但在很多实际问题中，**模型未知 (model-free)**，智能体只能通过与环境交互得到一条条经验轨迹。

蒙特卡洛方法 (Monte Carlo, MC) 的基本思想是：**用经验平均代替数学期望**。对于给定策略 π ，状态值函数定义为：

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right].$$

当模型未知时，无法像动态规划那样直接对所有后继状态求期望，因此可以通过多次采样 episode，记录状态 s 出现后的实际回报 G_t ，再取平均来估计 $v_{\pi}(s)$ 。

一次完整实验 (episode) 可以表示为：

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T.$$

从时刻 t 开始的回报为：

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T.$$

蒙特卡洛方法必须等一个 episode 结束后才能计算回报，因此它主要用于有终止状态的幕式任务。

4.2 蒙特卡洛策略评估

蒙特卡洛策略评估的目标是：在给定策略 π 的情况下，用采样回报估计 $v_{\pi}(s)$ 或 $q_{\pi}(s, a)$ 。

对于状态 s ，假设在多次 episode 中观察到它之后的回报为：

$$G_1(s), G_2(s), \dots, G_N(s).$$

则可以用经验平均估计状态值函数：

$$v(s) = \frac{G_1(s) + G_2(s) + \dots + G_N(s)}{N(s)}.$$

根据大数定律，当访问次数趋于无穷时：

$$v(s) \rightarrow v_{\pi}(s), \quad N(s) \rightarrow \infty.$$

Remark: 当只对一个点的值感兴趣，或者只对特定区域感兴趣时，蒙特卡洛方法效率很高。原因是 MC 可以只从关心的状态或区域采样，用这些 episode 的回报平均值估计价值；而动态规划通常需要依赖模型并反复扫描较大的状态空间。

4.2.1 首次访问 MC 与每次访问 MC

根据一条 episode 中同一个状态出现多次时如何处理，可以分为两类：

- **首次访问 MC (first-visit MC)**：每条 episode 中只使用状态 s 第一次出现后的回报。
- **每次访问 MC (every-visit MC)**：每条 episode 中状态 s 每次出现后的回报都用于平均。

二者在访问次数足够多时都可以收敛到真实值函数。区别在于：首次访问 MC 每条轨迹对同一状态只贡献一次样本；每次访问 MC 会利用同一条轨迹中该状态的所有访问样本。

4.3 蒙特卡洛评估行为值函数

在模型已知时，如果知道状态值函数 $v_\pi(s)$ ，可以通过模型计算动作值函数：

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s').$$

但在模型未知时，无法使用 $P_{ss'}^a$ 做上述计算。因此，**model-free** 情况下更重要的是直接估计行为值函数 $q_\pi(s, a)$ 。

行为值函数定义为：

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right].$$

MC 方法对每个状态-动作对 (s, a) 记录其出现后的回报，并取平均：

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s, a)} G_i(s, a).$$

这点非常关键：没有模型时，仅知道状态价值通常不足以改进策略，必须显式估计每个动作的价值。

4.4 MC 与 DP 策略评估的区别

蒙特卡洛评估和动态规划评估都用于估计值函数，但二者有明显区别：

- **是否需要模型**：DP 需要状态转移概率和回报函数；MC 不需要模型，只需要采样 episode。
- **是否 bootstrap**：DP 使用后继状态当前估计值更新当前状态，是 bootstrap 方法；MC 直接使用完整真实回报 G_t ，不依赖其他状态当前估计是否准确。
- **更新时机**：DP 可以一步步扫描状态更新；MC 通常要等 episode 结束后才能计算回报。
- **适用特点**：当只关心某个状态或某个区域的价值时，MC 可以只从相关样本估计，不必遍历整个状态空间。

MC 的优点是不需要模型、估计单个状态时较直接；缺点是必须等完整回合结束，且样本方差可能较大。

4.5 蒙特卡洛策略改进

策略评估的目的是为了帮助改进策略。对于 MC 方法，通常先估计动作值函数 $Q(s, a)$ ，再对每个状态采用贪婪策略：

$$\pi(s) = \arg \max_a Q(s, a).$$

这与动态规划中的策略改进思想一致：在每个状态选择当前估计动作价值最大的动作。

但 MC 控制存在一个重要问题：为了准确估计 $Q(s, a)$ ，需要所有状态-动作对都被充分访问。如果某些动作从未被尝试，就无法知道它们是否更优。

4.6 探索型初始化

探索型初始化 (exploring starts) 是一种保证所有状态-动作对都能被访问的方法。它要求每个 episode 的初始状态 S_0 和初始动作 A_0 都有一定概率被随机选到。

探索型初始化下的 MC 控制流程可以概括为：

1. 对所有 $s \in S, a \in A(s)$ 初始化 $Q(s, a)$ 和策略 $\pi(s)$ ；
2. 随机选择初始状态 S_0 和初始动作 A_0 ；
3. 从 (S_0, A_0) 开始按照当前策略生成一条 episode；
4. 对 episode 中出现的每个 (s, a) ，计算其首次出现后的回报 G ；
5. 将 G 加入 $Returns(s, a)$ ，并令 $Q(s, a)$ 等于回报平均值；
6. 对 episode 中出现的每个状态做策略改进：

$$\pi(s) = \arg \max_a Q(s, a).$$

探索型初始化的作用是：通过随机初始状态和初始动作，保证每个状态-动作对都有机会被访问。但在实际任务中，很多环境并不能任意指定初始状态和初始动作，因此需要更一般的探索策略。

4.7 无探索型初始化与永久温和策略

如果不能使用探索型初始化，就需要让策略本身保持探索性。课件中给出的条件是：策略必须是永久温和的，即对任意状态和动作都有：

$$\pi(a|s) > 0.$$

这保证每个动作始终有非零概率被选择，从而状态-动作对有机会被持续访问。

典型做法是使用 ϵ -soft 策略。设 $|A(s)|$ 是状态 s 下动作数，则：

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & a = \arg \max_a Q(s, a), \\ \frac{\epsilon}{|A(s)|}, & a \neq \arg \max_a Q(s, a). \end{cases}$$

该策略既倾向于选择当前最优动作，又给其他动作保留探索概率，因此能在探索和利用之间折中。

4.8 On-policy MC

On-policy 指的是：产生数据的策略与被评估和被改进的策略是同一个策略。在 on-policy MC 中，通常用 ε -soft 策略生成 episode，同时也评估和改进这个 ε -soft 策略。

On-policy MC 控制流程为：

1. 初始化 $Q(s, a)$ 、 $Returns(s, a)$ 和 ε -soft 策略 π ；
2. 按照当前策略 π 生成一条 episode；
3. 对 episode 中出现的每个 (s, a) ，计算回报 G ；
4. 用 G 的平均值更新 $Q(s, a)$ ；
5. 根据新的 $Q(s, a)$ 更新 ε -soft 策略。

On-policy 的特点是：一个策略既负责探索，也负责学习和改进。因此它必须在探索和利用之间妥协，最终学到的通常是 ε -soft 意义下的较优策略，而不是完全贪婪策略。

4.9 Off-policy MC

学习算法的困境是：需要以最优的动作采样，但同时又需要探索所有的动作。On-policy 方法在探索和利用中进行了妥协，使用 ε -soft 策略。Off-policy 方法通过引入两个策略解决这一矛盾：

- 目标策略 (target policy) π ：需要被学习和改进的策略，可以是贪婪策略。
- 行为策略 (behavior policy) μ ：用来产生样本的策略，负责探索，通常是 ε -soft 策略。

Off-policy 的核心思想是：用行为策略 μ 生成的数据，去学习目标策略 π 的价值。

为了保证行为策略产生的数据能够用于评估目标策略，需要满足覆盖性条件：

$$\pi(a|s) > 0 \Rightarrow \mu(a|s) > 0.$$

也就是说，只要目标策略可能选择某个动作，行为策略也必须有非零概率选择该动作。

4.10 重要性采样

Off-policy MC 中，样本来自行为策略 μ ，但我们想估计目标策略 π 下的期望。因此需要用**重要性采样 (importance sampling)** 修正分布差异。

4.10.1 一般重要性采样

一般地，若想计算某个函数在分布 $p(z)$ 下的期望：

$$\mathbb{E}_p[f] = \int f(z)p(z)dz,$$

但实际样本来自另一个分布 $q(z)$ ，则可以写成：

$$\mathbb{E}_p[f] = \int f(z)\frac{p(z)}{q(z)}q(z)dz.$$

定义重要性权重：

$$\omega_n = \frac{p(z_n)}{q(z_n)}.$$

则普通重要性采样估计为：

$$\mathbb{E}_p[f] \approx \frac{1}{N} \sum_{n=1}^N \omega_n f(z_n).$$

普通重要性采样是无偏估计，但方差可能很大，甚至可能出现无穷方差。当某个样本在 q 下概率很小、在 p 下概率较大时，权重 ω_n 会非常大，从而让估计不稳定。

为了降低方差，可以使用**加权重要性采样**：

$$\mathbb{E}_p[f] \approx \sum_{n=1}^N \frac{\omega_n}{\sum_{m=1}^N \omega_m} f(z_n).$$

加权重要性采样将权重归一化，通常更稳定，但会引入一定偏差。

4.10.2 MC 中的重要性权重

在 MC 中，从时刻 t 到终止时刻 T 的轨迹在目标策略 π 下的概率为：

$$\Pr(A_t, S_{t+1}, \dots, S_T) = \prod_{k=t}^{T-1} \pi(A_k|S_k) p(S_{k+1}|S_k, A_k).$$

行为策略 μ 下对应轨迹概率为：

$$\prod_{k=t}^{T-1} \mu(A_k|S_k) p(S_{k+1}|S_k, A_k).$$

因此，从时刻 t 到终止时刻 T 的重要性权重为：

$$\rho_t^T = \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k) p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} \mu(A_k|S_k) p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}.$$

这里环境转移概率会相互抵消，因此只需要策略概率之比。

这说明即使环境模型未知，也可以计算 **off-policy MC** 的重要性权重。因为我们不需要知道 $p(S_{k+1}|S_k, A_k)$ ，只需要知道目标策略和行为策略对实际动作给出的概率。

4.10.3 状态访问时刻集合

课件中的示意图如下：

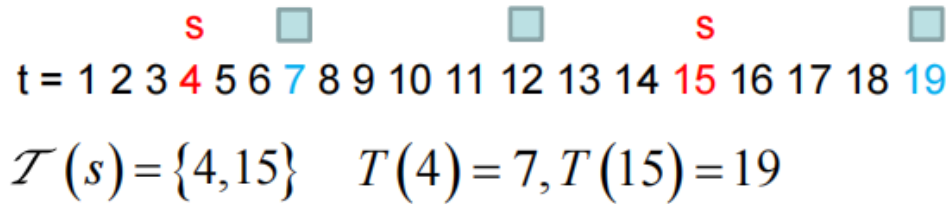


图 4.1: MC 重要性采样中的状态访问时刻示意图

课件中用 $\mathcal{T}(s)$ 表示状态 s 被访问过的所有时刻集合。例如，如果状态 s 在第 4 和第 15 个时刻被访问，则：

$$\mathcal{T}(s) = \{4, 15\}.$$

用 $T(t)$ 表示从时刻 t 开始之后对应 episode 的终止时刻。例如：

$$T(4) = 7, \quad T(15) = 19.$$

那么 G_t 表示从时刻 t 到终止时刻 $T(t)$ 的回报， $\rho_t^{T(t)}$ 表示从时刻 t 到该终止时刻的策略概率比。

4.10.4 普通重要性采样估计

普通重要性采样估计为：

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_t^{T(t)} G_t}{|\mathcal{T}(s)|}.$$

这个公式的含义是：每次访问状态 s 后得到一个回报 G_t ，但该回报来自行为策略 μ ，所以需要乘上 $\rho_t^{T(t)}$ ，把它修正为目标策略 π 下的估计。

4.10.5 加权重要性采样估计

加权重要性采样估计为：

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_t^{T(t)} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_t^{T(t)}}.$$

与普通重要性采样不同，加权重要性采样的分母不是访问次数 $|\mathcal{T}(s)|$ ，而是权重和：

$$\sum_{t \in \mathcal{T}(s)} \rho_t^{T(t)}.$$

这相当于对所有样本的权重做归一化。权重越大，说明该轨迹在目标策略下越可能出现，对最终估计的影响也越大。

普通重要性采样无偏但方差可能很大；加权重要性采样通常方差更小、更稳定，但会引入一定偏差。

因此，4.10 的核心逻辑是：

- **Off-policy 的困难：**数据来自行为策略 μ ，但要估计目标策略 π 。
- **重要性采样的作用：**用轨迹概率比 ρ 修正分布差异。
- **MC 中的关键简化：**环境转移概率在概率比中抵消，所以不需要环境模型。
- **普通重要性采样：**无偏，但方差可能很大。
- **加权重要性采样：**更稳定，但有一定偏差。

4.11 Off-policy Every-visit MC 控制

课件最后给出了 off-policy every-visit MC 控制算法。其思想是用行为策略 μ 产生 episode，用重要性采样加权更新目标策略 π 的动作值函数。

初始化：

$$Q(s, a) \leftarrow \text{任意}, \quad C(s, a) \leftarrow 0,$$

并令目标策略 π 为关于 Q 的贪婪策略。

每次利用行为策略 μ 生成一条 episode:

$$S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T.$$

然后从后向前更新:

$$G \leftarrow \gamma G + R_{t+1},$$

$$C(S_t, A_t) \leftarrow C(S_t, A_t) + W,$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)].$$

随后更新目标策略:

$$\pi(S_t) \leftarrow \arg \max_a Q(S_t, a).$$

如果当前动作不是目标策略会选择的贪婪动作, 即:

$$A_t \neq \pi(S_t),$$

则停止本条 episode 的反向更新; 否则更新重要性权重:

$$W \leftarrow W \frac{1}{\mu(A_t|S_t)}.$$

该算法体现了 off-policy MC 的核心: 行为策略负责探索, 目标策略保持贪婪并不断改进, 二者通过重要性采样连接起来。

4.12 蒙特卡洛方法小结

蒙特卡洛强化学习方法的主线可以概括为:

- **MC 策略评估:** 用 episode 回报的经验平均估计 $v_\pi(s)$ 或 $q_\pi(s, a)$ 。
- **First-visit 与 every-visit:** 分别使用首次访问或每次访问后的回报样本。
- **MC 控制:** 先估计 $Q(s, a)$, 再用贪婪或 ε -soft 策略改进。
- **On-policy:** 同一个策略既采样又被评估和改进。
- **Off-policy:** 行为策略负责探索, 目标策略负责学习最优行为。
- **重要性采样:** 用策略概率比修正 off-policy 数据分布差异。

与动态规划相比, MC 方法的最大特点是**不需要环境模型**; 与后续时间差分方法相比, MC 方法的特点是不进行 **bootstrap**, 必须等完整 episode 结束后才能更新。

第五章 时间差分及 DQN

5.1 时间差分学习的基本思想

时间差分学习 (Temporal-Difference Learning, TD) 结合了蒙特卡洛方法和动态规划方法的思想。它像 MC 一样不需要环境模型，可以通过采样学习；又像 DP 一样使用当前值函数估计进行自举更新。

DP、MC 和 TD 的区别可以概括为：

- **DP**：使用模型进行期望更新，需要知道 $P_{ss'}^a$ 和 R_s^a 。
- **MC**：使用完整 episode 的实际回报 G_t 更新，不需要模型，但必须等一个 episode 结束。
- **TD**：只等待一个时间步，用 $R_{t+1} + \gamma V(S_{t+1})$ 作为目标更新当前状态值。

MC 的增量式更新为：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)].$$

最简单的 TD 方法 TD(0) 的更新为：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

其中：

$$R_{t+1} + \gamma V(S_{t+1})$$

称为 **TD 目标**，而：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

称为 **TD 误差**。

5.2 TD 的偏差与方差

MC 使用完整回报 G_t ，它是 $v_\pi(S_t)$ 的无偏估计，但由于依赖一整条轨迹中的随机动作、状态转移和回报，因此方差较大。

TD 使用：

$$R_{t+1} + \gamma V(S_{t+1})$$

作为目标。由于 $V(S_{t+1})$ 本身是估计值，所以 TD 目标一般是**有偏的**；但它只依赖一个时间步的随机性，因此方差通常比 MC 小。

因此，TD 方法体现了强化学习中常见的**偏差-方差折中**：

- MC：低偏差，高方差；
- TD：有一定偏差，但方差较低，更新更及时。

5.3 TD 学习的优势

时间差分方法有三个重要优势：

- **不需要环境模型：**不需要状态转移概率、回报函数模型或下一状态概率分布。
- **不需要等到一个 episode 结束：**相比 MC，TD 只需要等待一个时间步即可更新。
- **可以在线学习 (online)：**智能体边与环境交互边更新值函数。

这些特点使 TD 成为后续 Sarsa、Q-learning 和 DQN 的基础。

5.4 Sarsa: On-policy TD 控制

Sarsa 是一种 on-policy TD 控制算法。它学习动作值函数 $Q(s, a)$ ，并且行为策略和评估策略是同一个策略，通常都是 ε -greedy 策略。

Sarsa 名称来自一次更新所用的数据序列：

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}.$$

其更新公式为：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Sarsa 的核心特点是：**下一动作 A_{t+1} 是按照当前行为策略真实选择出来的动作。**因此它评估的是当前 ε -greedy 策略本身。

算法流程：

1. 初始化 $Q(s, a)$ ，给定学习率 α 和折扣因子 γ ；
2. 在初始状态 s 根据 ε -greedy 策略选择动作 a ；
3. 执行动作 a ，得到回报 r 和下一状态 s' ；
4. 在 s' 根据同一策略选择下一动作 a' ；
5. 用 $r + \gamma Q(s', a')$ 更新 $Q(s, a)$ ；
6. 令 $s = s', a = a'$ ，重复直到终止状态。

5.4.1 Sarsa 课件伪代码

1. 初始化 $Q(s, a)$ ，对任意 $s \in S, a \in A(s)$ ；给定参数 α, γ 。
2. Repeat:
 - (a) 给定起始状态 s ，并根据 ε -greedy 策略在状态 s 选择动作 a 。
 - (b) Repeat (对于一个 episode 中的每一步):
 - i. 根据 ε -greedy 策略在状态 s 选择动作 a ，得到回报 r 和下一个状态 s' ；在状态 s' 根据 ε -greedy 策略得到动作 a' 。

ii. 更新动作值函数:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

iii. 令 $s = s'$, $a = a'$ 。

(c) Until s 是终止状态。

3. Until 所有的 $Q(s, a)$ 收敛。

4. 输出最终策略:

$$\pi(s) = \arg \max_a Q(s, a).$$

Sarsa 中, 行动策略和评估策略都是 ε -greedy 策略, 因此它是 on-policy 方法。

5.5 Q-learning: Off-policy TD 控制

Q-learning 是一种 **off-policy TD 控制算法**。它的行为策略通常是 ε -贪婪策略, 用于保证探索; 但它更新时使用的是下一状态的最大动作值, 因此目标策略是贪婪策略。

Q-learning 更新公式为:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)].$$

与 Sarsa 的区别在于:

- **Sarsa**: 使用真实选择的下一动作 A_{t+1} , 更新目标为 $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ 。
- **Q-learning**: 使用下一状态的最优动作值, 更新目标为 $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$ 。

因此, Sarsa 学习的是当前探索策略的价值, Q-learning 则直接逼近最优动作值函数 $q_*(s, a)$ 。

5.5.1 Q-learning 课件伪代码

1. 初始化 $Q(s, a)$, 对任意 $s \in S$, $a \in A(s)$; 给定参数 α, γ 。

2. Repeat:

(a) 给定起始状态 s , 并根据 ε -greedy 策略在状态 s 选择动作 a 。

(b) Repeat (对于一幕中的每一步):

i. 根据 ε -greedy 策略在状态 s_t 选择动作 a_t , 得到回报 r_t 和下一个状态 s_{t+1} 。

ii. 更新动作值函数:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)].$$

iii. 令 $s = s'$, 并继续按照行为策略选择动作。

(c) Until s 是终止状态。

3. Until 所有的 $Q(s, a)$ 收敛。

4. 输出最终策略:

$$\pi(s) = \arg \max_a Q(s, a).$$

Q-learning 的行动策略为 ε -greedy 策略, 目标策略为贪婪策略, 因此它是 off-policy 方法。

5.6 Expected Sarsa

Expected Sarsa 将 Sarsa 中的下一动作采样值替换为策略下的期望动作值：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

与普通 Sarsa 相比，它消除了下一动作随机采样带来的方差，但需要计算对动作的期望。

5.7 Double Q-learning

Q-learning 中的 max 操作容易带来**最大化偏差**：如果动作值估计中存在噪声，取最大值会倾向于高估真实价值。

Double Q-learning 的思路是使用两个动作值函数 Q_1 和 Q_2 ，将**动作选择**和**动作评估**分开。例如更新 Q_1 时：

$$a^* = \arg \max_a Q_1(S_{t+1}, a),$$

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha [R_{t+1} + \gamma Q_2(S_{t+1}, a^*) - Q_1(S_t, A_t)].$$

这样可以缓解普通 Q-learning 中由最大化操作带来的过估计问题。

第六章 函数逼近及 DQN

6.1 为什么需要函数逼近

表格型强化学习需要为每个状态-动作对存储一个值：

$$Q(s, a).$$

如果状态数为 $|S|$ 、动作数为 $|A|$ ，则需要存储 $|S| \cdot |A|$ 个值。

当状态空间连续、状态数量巨大或状态由图像表示时，表格方法会遇到两个问题：

- **存储和计算不可行**：状态空间可能接近无穷大。
- **缺乏泛化能力**：很多状态可能只出现一次，无法依赖重复访问来学习。

因此，需要用函数逼近表示值函数：

$$\hat{q}(s, a; \theta),$$

其中 θ 是需要学习的参数。函数逼近的目标是：用有限参数表示大量状态-动作对的价值，并对未见过的状态具有泛化能力。

6.2 函数逼近方法分类

函数逼近方法可以分为：

- **参数化方法**：用有限参数表示函数，例如线性函数、神经网络、决策树、模糊网络等。
- **非参数方法**：例如核方法、高斯过程回归等。

参数化方法又可以分为：

- **线性参数化**：利用人工设计的特征或基函数。
- **非线性参数化**：使用神经网络等复杂函数逼近器。

基于函数逼近的强化学习可以理解为：

$$\text{强化学习} + \text{泛化方法}.$$

需要注意的是，强化学习中的函数逼近比普通监督学习更困难，因为存在非静态目标、自举更新和延迟回报。

6.3 值函数逼近的目标

在表格型强化学习中，每个状态-动作对都有一个单独的表格值 $Q(s, a)$ 。更新时只需要把对应格子的值向某个 **backup 值** 靠近。例如 DP 中的期望 backup 为：

$$Q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') Q_{\pi}(s', a').$$

但是在大规模或连续状态空间中，无法为每个 (s, a) 单独存表，因此使用函数逼近：

$$\hat{q}(S_t, A_t; \theta).$$

这里 θ 是所有状态-动作对共享的一组参数。也就是说，改变一次参数 θ ，不只会改变当前样本 (S_t, A_t) 的估计值，也会影响其他状态-动作对的估计值。

函数逼近的训练目标可以类比监督学习：输入是 (S_t, A_t) ，模型输出是 $\hat{q}(S_t, A_t; \theta)$ ，而标签由强化学习算法构造出来，称为 **backup 值** 或目标值 U_t 。于是损失函数可以写成：

$$L(\theta) = \frac{1}{2} [U_t - \hat{q}(S_t, A_t; \theta)]^2.$$

关键问题变成：如何构造目标值 U_t ？不同强化学习方法给出的 backup 值不同：

- **MC 目标**：使用完整 episode 后得到的真实回报。

$$U_t = G_t.$$

它不使用自举，目标较稳定但必须等一幕结束。

- **TD 目标**：使用一步回报加下一状态估计值。

$$U_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \theta).$$

它是自举目标，可以在线更新，但目标本身依赖当前估计。

- **Q-learning 目标**：使用一步回报加下一状态最大动作值。

$$U_t = R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a; \theta).$$

它直接朝最优动作值函数逼近。

因此，函数逼近下的值函数估计过程可以理解为：

1. 与环境交互得到样本 $(S_t, A_t, R_{t+1}, S_{t+1})$ ；
2. 根据 MC、TD 或 Q-learning 构造目标值 U_t ；
3. 让当前估计 $\hat{q}(S_t, A_t; \theta)$ 向目标值 U_t 靠近；
4. 更新参数 θ 后，相关状态-动作对的估计值都会随之变化。

参数更新通常使用随机梯度下降：

$$\theta \leftarrow \theta + \alpha [U_t - \hat{q}(S_t, A_t; \theta)] \nabla_{\theta} \hat{q}(S_t, A_t; \theta).$$

这体现了课件中“强化学习是在线学习”的含义：数据是一边采集一边用于更新的；同时，由于策略和值函数都在变化，目标 U_t 也会随训练过程变化，所以强化学习中的目标函数通常是**非静态目标函数**。这也是函数逼近强化学习比普通监督学习更难的原因之一。非静态目标函数：增量式学习方法和批方法

6.4 半梯度方法

对于 MC 方法，目标 $U_t = G_t$ 是一条 episode 结束后得到的实际回报，不依赖当前参数 θ 。此时可以把问题近似看成普通监督学习：

$$L(\theta) = \frac{1}{2} [G_t - \hat{v}(S_t; \theta)]^2.$$

但对于 TD(0)、DP backup、Q-learning 等带有 bootstrapping 的方法，目标值本身也包含当前估计值。例如 TD(0) 的目标为：

$$U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; \theta).$$

这个目标中的 $\hat{v}(S_{t+1}; \theta)$ 依赖当前参数 θ 。严格来说，如果对损失函数

$$L(\theta) = \frac{1}{2} [U_t - \hat{v}(S_t; \theta)]^2$$

求完整梯度，不仅要考虑 $\hat{v}(S_t; \theta)$ 对 θ 的影响，还要考虑 U_t 对 θ 的影响。

半梯度方法 (semi-gradient) 的做法是：只考虑参数 θ 对当前估计值 $\hat{v}(S_t; \theta)$ 的影响，而忽略目标 U_t 对参数的影响。也就是说，更新时把 U_t 暂时当作一个固定标签。

这样做的原因是：

- **计算简单**：不需要对 TD 目标中的下一状态估计值再求梯度。
- **适合在线学习**：每来一个样本就可以更新一次参数。
- **学习速度快**：实际强化学习算法中经常采用这种近似。
- **可用于连续状态空间**：配合函数逼近器，可以处理表格方法无法处理的大规模问题。

例如半梯度 TD 更新可写为：

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \hat{v}(S_{t+1}; \theta) - \hat{v}(S_t; \theta)] \nabla_{\theta} \hat{v}(S_t; \theta).$$

其中括号内：

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; \theta) - \hat{v}(S_t; \theta)$$

是 TD 误差。因此半梯度 TD(0) 也可以写成：

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \hat{v}(S_t; \theta).$$

6.4.1 半梯度 TD(0) 值函数评估流程

课件中的半梯度 TD(0) 值函数评估算法可以理解为：

1. 输入要评估的策略 π ，以及一个可微值函数逼近器 $\hat{v}(s; \theta)$ 。
2. 初始化参数 θ 。
3. Repeat:
 - (a) 初始化状态 S 。
 - (b) Repeat (对于一幕中的每一步):
 - i. 按策略 π 选择动作 A ;

- ii. 执行动作, 观测回报 R 和下一状态 S' ;
- iii. 计算 TD 误差:

$$\delta = R + \gamma \hat{v}(S'; \theta) - \hat{v}(S; \theta).$$

- iv. 更新参数:

$$\theta \leftarrow \theta + \alpha \delta \nabla_{\theta} \hat{v}(S; \theta).$$

- v. 令 $S = S'$ 。

(c) Until S' 是终止状态。

6.4.2 半梯度 Sarsa

对于动作值函数, 也可以使用半梯度方法。设可微动作值函数为:

$$\hat{q}(s, a; \theta).$$

半梯度 Sarsa 的目标为:

$$U_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \theta),$$

更新为:

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \theta) - \hat{q}(S_t, A_t; \theta)] \nabla_{\theta} \hat{q}(S_t, A_t; \theta).$$

其流程是:

1. 输入可微动作值函数 $\hat{q}(s, a; \theta)$, 初始化参数 θ ;
2. 每一幕开始时初始化状态-动作对 (S, A) ;
3. 执行动作 A , 得到 R, S' ;
4. 如果 S' 是终止状态, 则用终止回报更新并进入下一幕;
5. 否则按 soft 策略在 S' 选择 A' ;
6. 用上式更新参数, 并令 $S = S', A = A'$ 。

半梯度方法的关键理解是: 目标值中虽然含有当前参数估计, 但更新时只把当前状态的估计值往目标值方向推, 而不追踪目标值本身随参数变化的梯度。

6.5 线性值函数逼近

线性逼近假设值函数可以表示为特征的线性组合:

$$\hat{v}(s; \theta) = \phi(s)^T \theta,$$

或:

$$\hat{q}(s, a; \theta) = \phi(s, a)^T \theta.$$

其中, $\phi(s)$ 或 $\phi(s, a)$ 是特征函数。常用基函数包括:

- 多项式基函数;

- 傅里叶基函数；
- 径向基函数；
- 固定稀疏表示。

线性逼近的优点是优化问题相对简单，在线性情况下通常只有一个全局最优解。

表格型强化学习可以看作线性函数逼近的特殊情况：每个状态-动作对对应一个 one-hot 特征，此时参数向量中的每个元素就是表格中的一个 $Q(s, a)$ 。

6.6 批方法

增量式方法计算简单，但样本效率可能不高。批方法利用一批经验数据整体求解最优参数。常见批方法包括：

- 最小二乘蒙特卡洛 (LSMC)：用回报 G_t 拟合值函数。
- 最小二乘时间差分 (LSTD)：利用 TD 误差构造最小二乘问题。
- LSTD(λ)：结合资格迹思想，利用多步信息进行批量估计。

批方法通常样本效率较高，但计算和存储成本也更大。

6.7 表格型强化学习是一种特殊的函数逼近

课件指出，表格型强化学习可以看作函数逼近方法的一种特殊形式。一般的线性动作值函数逼近写为：

$$Q(s, a) = \phi(s, a)^T \Theta.$$

其中， $\phi(s, a)$ 是状态-动作对 (s, a) 的特征向量， Θ 是参数向量。

在表格型强化学习中，每一个状态-动作对都有一个独立的表格元素。例如格子世界中有 100 个状态、4 个动作，则表格型动作值函数共有：

$$100 \times 4$$

个元素。此时可以把每一个状态-动作对看成一个 one-hot 特征：

- 当前状态-动作对对应的特征位置为 1；
- 其他所有状态-动作对对应的特征位置为 0。

这样， Θ 中的每一个参数就对应原表格中的一个 $Q(s, a)$ 。因此：

$$Q(s, a) = \phi(s, a)^T \Theta$$

在 one-hot 特征下等价于直接查表。

这个角度很重要：表格型方法不是函数逼近的对立面，而是函数逼近中最特殊、最简单的一种情况。区别在于，表格型方法没有泛化能力：一个状态-动作对的更新只改变表格中对应的一个值。而一般函数逼近中，多个状态-动作对共享参数，更新一个样本会影响许多相关状态的估计值。

6.8 固定稀疏表示

固定稀疏表示是一种用人工设计的稀疏特征来表示连续状态的方法。设状态空间有 n 个维度：

$$s = [s_1, \dots, s_n].$$

如果每个维度离散化为 d 个取值，则第 i 个维度可以用 d 个特征表示，记为：

$$\phi_i^1, \phi_i^2, \dots, \phi_i^d.$$

于是状态特征可以写成：

$$\phi(s) = [\phi_{11}, \phi_{12}, \dots, \phi_{1d}, \phi_{21}, \dots, \phi_{2d}, \dots, \phi_{n1}, \dots, \phi_{nd}]^T.$$

对于动作值函数，需要把状态特征扩展到状态-动作特征。若动作集合大小为 $|A|$ ，则可写为：

$$\phi(s, a) = [\phi^1(s), \dots, \phi^{|A|}(s)].$$

其中只有与当前动作 a 对应的那一块特征被激活，其他动作对应的特征为 0。

固定稀疏表示的特征数和参数数为：

$$d \times n \times |A|.$$

这说明它的特征数随状态维度 n 线性增长，而不是像完整离散化那样指数增长。

直观理解：

- 完整离散化会考虑所有维度取值组合，容易产生维数灾难；
- 固定稀疏表示把每个维度分开编码，特征向量中只有少量元素为 1；
- 它牺牲了一部分精细交互表达能力，但显著降低了表示规模。

因此，固定稀疏表示可以看作从表格型方法走向一般函数逼近的一种过渡：仍然使用人工特征，但已经开始利用特征共享来获得一定泛化能力。

6.9 神经网络值函数逼近

神经网络可以看作一种非线性参数化的基函数方法。前向传播用于计算输出值函数，反向传播用于计算梯度并更新参数。

6.9.1 前向神经网络的正向传播

课件中强调：神经网络可以看成是基函数参数化的一种方法。在线性函数逼近中，特征函数 $\phi(s)$ 通常由人工设计；而在神经网络中，隐藏层相当于自动学习出来的参数化基函数。

设第 l 层第 i 个神经元的加权和为 $z_i^{(l)}$ ，激活值为 $a_i^{(l)}$ 。以前一层输出为输入，则：

$$z_i^{(l)} = \sum_j W_{ij}^{(l-1)} a_j^{(l-1)} + b_i^{(l-1)},$$

$$a_i^{(l)} = f(z_i^{(l)}),$$

其中 W 是权重， b 是偏置， $f(\cdot)$ 是激活函数。

以一个三层前向神经网络为例，输入为 x ，第一隐层计算为：

$$z^{(2)} = W^{(1)}x + b^{(1)},$$

$$a^{(2)} = f(z^{(2)}).$$

输出层计算为：

$$z^{(3)} = W^{(2)}a^{(2)} + b^{(2)},$$

$$h_{w,b}(x) = a^{(3)} = f(z^{(3)}).$$

若网络共有 n_l 层，输出层也可写为：

$$y = z^{(n_l)} = W^{(n_l-1)}a^{(n_l-1)} + b^{(n_l-1)}.$$

在值函数逼近中，神经网络的输出可以是：

- **状态值函数**：输入状态 s ，输出 $\hat{v}(s; \theta)$ ；
- **动作值函数**：输入状态-动作对 (s, a) ，输出 $\hat{q}(s, a; \theta)$ ；
- **DQN 常用形式**：输入状态 s ，一次输出所有动作的 $Q(s, a; \theta)$ 。

因此，前向传播的作用就是：把状态或状态-动作特征逐层变换，最终得到值函数估计。

6.9.2 卷积神经网络

在图像输入任务中，卷积神经网络（CNN）常用于提取视觉特征：

- **卷积**：权值共享、稀疏连接，用于提取局部特征。
- **池化**：抽象特征表示，减少参数和计算量。

6.10 DQN

DQN（Deep Q-Network）是 **Q-learning** 与深度神经网络的结合。它使用神经网络近似动作值函数：

$$Q(s, a; \theta).$$

DQN 的基本框架仍然是 Q-learning，其 TD 目标为：

$$y = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta^-),$$

损失函数为：

$$L(\theta) = [y - Q(S_t, A_t; \theta)]^2.$$

其中， θ^- 表示目标网络参数。

DQN 的三个关键技巧是：

- **卷积神经网络**：用 CNN 从图像状态中学习值函数表示。
- **经验回放**：将经验存入 replay buffer，训练时随机采样 mini-batch，减弱样本相关性并提高数据利用率。
- **目标网络**：使用单独的目标网络计算 TD 目标，减小目标随当前网络快速变化导致的不稳定。

6.10.1 DQN 课件伪代码

DQN 可以看作把 Q-learning 中的表格 $Q(s, a)$ 替换为神经网络 $Q(\phi(s), a; \theta)$ ，其中 $\phi(s)$ 表示对原始状态序列进行预处理后的输入特征。课件中的伪代码可整理为：

1. 初始化经验回放池 D ，容量为 N 。
2. 随机初始化动作值函数网络 Q 的参数 θ 。
3. 初始化目标动作值函数网络 $\hat{Q}$ ，令其参数：

$$\theta^- = \theta.$$

4. For episode = 1, $\dots$, M :

- (a) 初始化状态序列：

$$s_1 = \{x_1\},$$

并进行预处理：

$$\phi_1 = \phi(s_1).$$

- (b) For $t = 1, \dots, T$:

- i. 以概率 ε 随机选择动作 a_t 。
- ii. 否则选择当前网络下动作值最大的动作：

$$a_t = \arg \max_a Q(\phi(s_t), a; \theta).$$

- iii. 在环境中执行动作 a_t ，观测回报 r_t 和新图像 x_{t+1} 。
- iv. 构造新状态序列：

$$s_{t+1} = s_t, a_t, x_{t+1},$$

并预处理：

$$\phi_{t+1} = \phi(s_{t+1}).$$

- v. 将转移样本存入经验回放池：

$$(\phi_t, a_t, r_t, \phi_{t+1}) \in D.$$

- vi. 从 D 中随机采样一个 mini-batch:

$$(\phi_j, a_j, r_j, \phi_{j+1}).$$

- vii. 构造 TD 目标：

$$y_j = \begin{cases} r_j, & \text{若在 } j+1 \text{ 步终止,} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-), & \text{否则.} \end{cases}$$

- viii. 对损失函数做梯度下降：

$$(y_j - Q(\phi_j, a_j; \theta))^2.$$

- ix. 每隔 C 步重置目标网络：

$$\hat{Q} = Q, \quad \theta^- = \theta.$$

这段伪代码中，行为策略是 ε -greedy 策略，用于产生数据；而 TD 目标中使用

$$\max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-)$$

对应贪婪目标策略。因此，DQN 继承了 Q-learning 的 **off-policy** 特点。

6.11 DQN 的常见变种

- **Double DQN**: 两个值函数逼近网络, 一个用来选择动作, 一个用来评估值函数。将动作选择和动作评估分开, 缓解 $\max$ 操作带来的过估计问题。
- **Prioritized DQN**: 经验回放时优先采样 TD 误差较大的样本, 提高学习效率。
- **Dueling Network**: 将值函数分解为状态价值 $V(s)$ 和优势函数 $A(s, a)$, 提升对动作差异不明显状态的估计能力。
- **DRQN**: 在 DQN 中加入循环神经网络, 用于部分可观测 MDP。
- **Rainbow**: 综合 Double DQN、优先经验回放、Dueling Network、多步回报、分布式价值等多个改进。

第七章 基于策略梯度的方法

7.1 为什么需要策略搜索

前面基于值函数的方法通常先学习 $Q(s, a)$ ，再通过：

$$\pi(s) = \arg \max_a Q(s, a)$$

得到策略。但当动作空间很大或连续时，求 $\arg \max$ 可能非常困难。

策略搜索方法直接参数化策略，并直接优化策略参数。设策略为：

$$\pi_{\theta}(a|s),$$

目标是寻找最优参数 θ ，使期望回报最大：

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_0].$$

策略搜索的优点包括：

- 可以自然处理连续动作空间；
- 参数化策略可以逼近复杂概率分布甚至近似确定性策略；
- 参数化的策略可以逼近任意概率分布，不受行为值函数的限制
- 策略是更简单的函数逼近，如 PID 控制；
- 策略参数化更容易加入先验知识。

7.2 策略搜索方法分类

策略搜索方法可分为：

- **无模型策略搜索**：不依赖环境模型，直接通过采样优化策略。
- **基于模型策略搜索**：先学习或利用环境模型，再优化策略。

也可以按策略形式分为：

- **随机策略**：在状态 s 下按概率分布采样动作。
- **确定性策略**：在状态 s 下直接输出动作。

常见方法包括策略梯度、EM 方法、路径积分方法、指导策略搜索、DDPG 等。

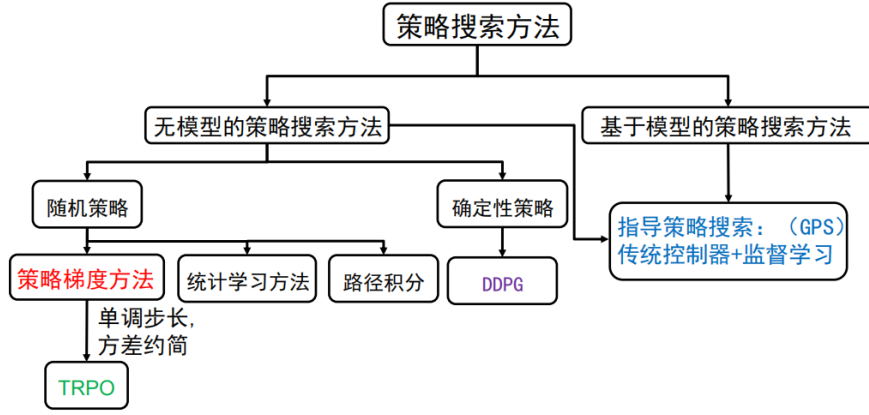


图 7.1: 策略搜索方法分类图

7.3 似然率策略梯度

课件中用 τ 表示一条状态-行为序列:

$$\tau = s_0, u_0, \dots, s_H, u_H.$$

对应的轨迹总回报写为:

$$R(\tau) = \sum_{t=0}^H R(s_t, u_t).$$

策略参数为 θ 时, 轨迹 τ 出现的概率记为 $P(\tau; \theta)$ 。因此目标函数可以写成所有可能轨迹回报的期望:

$$U(\theta) = \mathbb{E} \left(\sum_{t=0}^H R(s_t, u_t); \pi_\theta \right) = \sum_{\tau} P(\tau; \theta) R(\tau).$$

强化学习的目标就是寻找最优参数 θ , 使得

$$\max_{\theta} U(\theta) = \max_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau).$$

对目标函数关于参数求梯度:

$$\begin{aligned} \nabla_{\theta} U(\theta) &= \nabla_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} P(\tau; \theta) R(\tau). \end{aligned}$$

直接求 $\nabla_{\theta} P(\tau; \theta)$ 不方便, 因此引入似然率技巧:

$$\nabla_{\theta} \log P(\tau; \theta) = \frac{\nabla_{\theta} P(\tau; \theta)}{P(\tau; \theta)},$$

也就是

$$\nabla_{\theta} P(\tau; \theta) = P(\tau; \theta) \nabla_{\theta} \log P(\tau; \theta).$$

代回梯度公式可得:

$$\begin{aligned} \nabla_{\theta} U(\theta) &= \sum_{\tau} P(\tau; \theta) \nabla_{\theta} \log P(\tau; \theta) R(\tau) \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P(\tau; \theta) R(\tau)]. \end{aligned}$$

这说明策略梯度可以用采样轨迹的经验平均来估计:

$$\nabla_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) R(\tau^{(i)}).$$

从重要性采样的视角推导

课件还从重要性采样角度解释了同一个公式。若数据由旧策略参数 θ_{old} 产生，则可以把新参数 θ 下的目标写为：

$$U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{P(\tau|\theta)}{P(\tau|\theta_{\text{old}})} R(\tau) \right].$$

因此导数为：

$$\nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{\nabla_{\theta} P(\tau|\theta)}{P(\tau|\theta_{\text{old}})} R(\tau) \right].$$

当在同分布处评价，也就是令 $\theta = \theta_{\text{old}}$ 时：

$$\begin{aligned} \nabla_{\theta} U(\theta)|_{\theta=\theta_{\text{old}}} &= \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{\nabla_{\theta} P(\tau|\theta)|_{\theta_{\text{old}}}}{P(\tau|\theta_{\text{old}})} R(\tau) \right] \\ &= \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\nabla_{\theta} \log P(\tau|\theta)|_{\theta_{\text{old}}} R(\tau) \right]. \end{aligned}$$

这不仅能推导出策略梯度公式，也说明可以构造新的损失函数：

$$U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{P(\tau|\theta)}{P(\tau|\theta_{\text{old}})} R(\tau) \right],$$

即用旧策略采样到的数据，通过概率比来评估新策略。

路径似然率的展开

上面的公式还需要计算 $\nabla_{\theta} \log P(\tau; \theta)$ 。对一条样本轨迹 $\tau^{(i)}$ ，其概率可以分解为：

$$P(\tau^{(i)}; \theta) = \prod_{t=0}^H P(s_{t+1}^{(i)} | s_t^{(i)}, u_t^{(i)}) \cdot \pi_{\theta}(u_t^{(i)} | s_t^{(i)}).$$

其中 $P(s_{t+1} | s_t, u_t)$ 是环境动力学， $\pi_{\theta}(u_t | s_t)$ 是策略。取对数后：

$$\log P(\tau^{(i)}; \theta) = \sum_{t=0}^H \log P(s_{t+1}^{(i)} | s_t^{(i)}, u_t^{(i)}) + \sum_{t=0}^H \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}).$$

由于环境动力学不含策略参数 θ ，所以关于 θ 求导时，动力学部分为 0：

$$\begin{aligned} \nabla_{\theta} \log P(\tau^{(i)}; \theta) &= \nabla_{\theta} \left[\sum_{t=0}^H \log P(s_{t+1}^{(i)} | s_t^{(i)}, u_t^{(i)}) + \sum_{t=0}^H \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right] \\ &= \sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}). \end{aligned}$$

因此最终的路径似然率策略梯度估计为：

$$\nabla_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m R(\tau^{(i)}) \sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}).$$

直观理解： $\nabla_{\theta} \log P(\tau^{(i)}; \theta)$ 只改变经验路径被当前策略选中的概率，而不改变已经发生的路径本身；乘上 $R(\tau^{(i)})$ 后，算法就会增加高回报路径的概率，减小低回报路径的概率。同时，由于动力学项在求导中消失，路径似然率推导不要求环境动力学模型已知，只需要知道当前策略对动作给出的概率。

7.4 常见策略表示

离散动作空间中，常用 softmax 策略：

$$\pi_{\theta}(a|s) = \frac{\exp(h_{\theta}(s, a))}{\sum_b \exp(h_{\theta}(s, b))}.$$

连续动作空间中，常用高斯随机策略：

$$a = \mu_{\theta}(s) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

其中，均值 $\mu_{\theta}(s)$ 可以由线性函数、径向基函数或神经网络表示；方差参数控制探索强度。

7.5 随机策略梯度的推导及解释

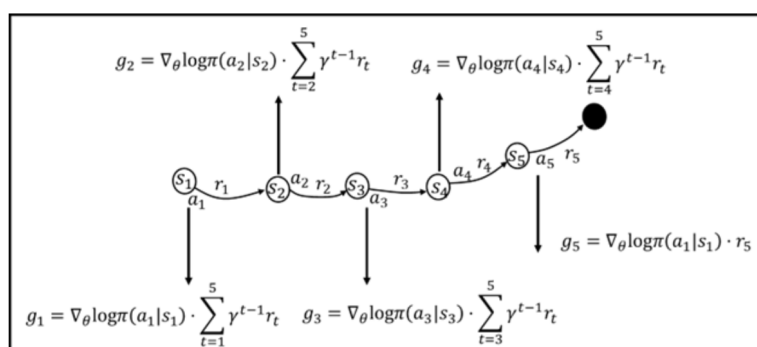


随机策略梯度的推导及解释



$$\nabla_{\theta} U(\theta) \approx \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \left(\sum_{k=t}^{H-1} (R(s_k^{(i)}) - b) \right)$$

状态层面解释策略梯度：



(不知道该说什么，这张 ppt 来的很莫名其妙，没有推导过程，看看得了)

7.6 REINFORCE

REINFORCE 是典型的蒙特卡洛策略梯度算法。课件首先给出单步更新的核心形式：

$$\theta_{t+1} := \theta_t + \alpha G_t \nabla_{\theta} \log \pi(a_t | s_t; \theta_t).$$

若把一幕 episode 中每个时刻的更新写在一起，课件伪代码对应的形式为：

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \gamma^t G_t \nabla_{\theta} \ln \pi(A_t | S_t, \theta).$$

其中

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k.$$

也就是说，当前时刻 t 的策略梯度只乘以从该时刻之后实际得到的折扣回报。

REINFORCE 伪代码：

1. 输入可微的策略参数化形式 $\pi(a|s, \theta)$;
2. 设定算法参数: 步长 $\alpha > 0$;
3. 初始化策略参数 $\theta \in \mathbb{R}^{d'}$, 例如初始化为 0;
4. 对每一幕 episode 循环执行:

(a) 按当前策略 $\pi(\cdot|\cdot, \theta)$ 生成一条轨迹

$$S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T;$$

(b) 对该幕中的每一步 $t = 0, 1, \dots, T-1$, 计算

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k,$$

并更新

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla_{\theta} \ln \pi(A_t | S_t, \theta).$$

REINFORCE 不需要值函数模型, 但因为使用完整回报进行蒙特卡洛估计, 梯度估计无偏但方差通常很大。

7.7 基线与方差减小

由似然率策略梯度可知, 一条轨迹上的经验估计可以写为:

$$\nabla_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) R(\tau^{(i)}).$$

进一步展开路径似然率, 有:

$$\nabla_{\theta} U(\theta) \approx \frac{1}{m} \sum_{i=1}^m \left(\sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right) R(\tau^{(i)}).$$

这个估计是无偏的, 但由于完整轨迹回报 $R(\tau)$ 波动很大, 方差通常很大。

一种常见做法是在回报中减去一个基线 b :

$$\nabla_{\theta} U(\theta) \approx \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) (R(\tau^{(i)}) - b).$$

这样做不会改变梯度期望, 原因是基线项的期望为 0:

$$\begin{aligned} \mathbb{E} [\nabla_{\theta} \log P(\tau; \theta) b] &= \sum_{\tau} P(\tau; \theta) \nabla_{\theta} \log P(\tau; \theta) b \\ &= \sum_{\tau} P(\tau; \theta) \frac{\nabla_{\theta} P(\tau; \theta)}{P(\tau; \theta)} b \\ &= \sum_{\tau} \nabla_{\theta} P(\tau; \theta) b \\ &= \nabla_{\theta} \left(\sum_{\tau} P(\tau; \theta) b \right) \\ &= \nabla_{\theta} b = 0. \end{aligned}$$

因此，**减去基线不会引入偏差，只会改变方差**。直观上，基线相当于把“绝对回报”改成“相对回报”：如果 $R(\tau) > b$ ，则增强该轨迹中动作的概率；如果 $R(\tau) < b$ ，则削弱该轨迹中动作的概率。

课件还给出了常数基线的最优取值思路。令

$$X = \nabla_{\theta} \log P(\tau^{(i)}; \theta) (R(\tau^{(i)}) - b),$$

则希望选择 b 使 X 的方差尽量小。由于基线不改变期望，方差中的均值项与 b 无关，最终可得近似的最优常数基线：

$$b^* = \frac{\mathbb{E}_p \left[\left(\sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right)^2 R(\tau) \right]}{\mathbb{E}_p \left[\left(\sum_{t=0}^H \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right)^2 \right]}.$$

实际算法中一般不会精确计算这个 b^* ，而是用更容易估计的状态值函数作为基线。

修改值函数：只使用当前动作之后的回报。前面的写法把整条轨迹回报 $R(\tau)$ 都乘到每一个动作的梯度上，但这会把过去已经发生的奖励也分配给当前动作。课件强调：

$$\mathbb{E}_p [\partial_{\theta} \log \pi_{\theta}(u_t | x_t, t) r_j] = 0, \quad j < t.$$

含义是：当前动作 u_t 不可能影响过去时刻 $j < t$ 的奖励 r_j ，所以这些过去回报对当前动作的梯度贡献期望为 0。因此，可以把整条轨迹回报改成从当前时刻开始的后续回报：

$$\nabla_{\theta} U(\theta) \approx \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \left(\sum_{k=t}^{H-1} (R(s_k^{(i)}) - b) \right).$$

这一步的逻辑是**信用分配**：某个动作只应该根据它之后可能受到影响的奖励来更新，而不应该根据它之前已经发生的奖励来更新。

从奖励时刻的角度也可以写成：

$$\nabla_{\theta} U(\theta) \approx \frac{1}{m} \sum_{i=1}^m \sum_{j=0}^{H-1} \left(\sum_{t=0}^j \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) (r_j - b_j) \right).$$

这表示奖励 r_j 只向它之前以及当前的动作分配贡献，而不会影响未来动作。

在标准强化学习记号下，把从 t 时刻开始的后续折扣回报写为 G_t ，并令基线依赖状态但不依赖动作：

$$b(S_t) = V(S_t),$$

就得到更常见的方差减小形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) (G_t - V(S_t))].$$

其中 $G_t - V(S_t)$ 可以看作优势函数的估计，表示当前动作相对于该状态平均水平到底好多少。加入基线的核心作用是降低策略梯度估计的方差，而不改变其期望。

7.8 策略梯度定理

策略梯度定理给出更一般的形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim \rho_{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q_{\pi}(s, a)].$$

它说明在计算策略梯度时，可以用动作值函数 $Q_{\pi}(s, a)$ 评价动作好坏，而不需要显式求状态分布对参数的导数。

第八章 TRPO 及 PPO

8.1 为什么普通策略梯度需要改进

上一讲的 REINFORCE 和 Actor-Critic 都属于策略梯度方法。它们直接沿着估计出的梯度方向更新策略参数：

$$\theta \leftarrow \theta + \alpha \widehat{\nabla_{\theta} J(\theta)}.$$

这种形式直观，但有几个突出问题。

- **步长难以选择**：在学习过程中，状态分布、动作分布和回报分布都会随着策略变化而变化。固定学习率很难始终合适，步长过小学习慢，步长过大则可能使策略突然变差。
- **坏策略会破坏后续数据**：策略梯度是 on-policy 方法居多，一旦某次更新得到坏策略，之后采集到的数据也会变差，训练可能迅速崩溃。
- **容易过早确定化**：策略可能过早收敛到近似确定性策略，陷入局部最优并失去探索能力。

TRPO 和 PPO 的核心思想都是：**不要让新策略离旧策略太远**。TRPO 用 KL 散度构造显式的信赖域约束；PPO 用裁剪后的概率比给出更容易实现的近端更新。

8.2 性能差分与替代回报函数

设 $\tau = (s_0, a_0, \dots, s_H, a_H)$ 表示一条状态-动作序列，策略 π 的期望折扣回报为

$$\eta(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t) \right].$$

若要比旧策略 π 和新策略 $\tilde{\pi}$ ，直接比较 $\eta(\tilde{\pi})$ 很困难，因为新策略会改变整条轨迹的状态分布。课件先把真实性能差写成优势函数的形式。

优势函数定义为

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s) = \mathbb{E}_{s' \sim P(\cdot | s, a)} [r(s) + \gamma V_{\pi}(s') - V_{\pi}(s)].$$

它表示动作 a 相对旧策略 π 在状态 s 下平均动作的好坏。

8.2.1 性能差分恒等式

课件给出的等价替代形式为

$$\eta(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi}(s_t, a_t) \right].$$

证明的关键是把优势函数展开成 TD 误差并望远镜求和：

$$\begin{aligned}
 & \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi}(s_t, a_t) \right] \\
 &= \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t) + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)) \right] \\
 &= \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t) \right] + \mathbb{E}_{s_0} [-V_{\pi}(s_0)] \\
 &= \eta(\tilde{\pi}) - \eta(\pi).
 \end{aligned}$$

其中最后一步使用了

$$\eta(\pi) = \mathbb{E}_{s_0} [V_{\pi}(s_0)].$$

因此，新策略相对旧策略的性能提升，等于在新策略轨迹上累积旧策略优势函数的期望。

8.3 折扣状态分布与精确替代目标

定义策略 π 的折扣状态访问频率

$$\rho_{\pi}(s) = P(s_0 = s) + \gamma P(s_1 = s) + \gamma^2 P(s_2 = s) + \dots$$

利用它可以把上面的轨迹期望写成状态-动作求和：

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a|s) A_{\pi}(s, a).$$

这个式子是精确的，但问题在于 $\rho_{\tilde{\pi}}$ 由新策略产生，对 $\tilde{\pi}$ 严重依赖，优化起来很困难。

8.4 策略改善理论及其证明

如果在每个状态 s 上，新策略相对旧策略的期望优势都非负：

$$\sum_a \tilde{\pi}(a|s) A_{\pi}(s, a) \geq 0,$$

那么由上式可知 $\eta(\tilde{\pi}) \geq \eta(\pi)$ ，策略不会变差。

课件还给出了一步递推式证明。若对所有状态满足

$$V_{\pi}(s) \leq Q_{\pi}(s, \pi'(s)),$$

则

$$\begin{aligned}
 V_{\pi}(s) &\leq \mathbb{E} [R_{t+1} + \gamma V_{\pi}(s_{t+1}) \mid s_t = s, a_t = \pi'(s)] \\
 &= \mathbb{E}_{\pi'} [R_{t+1} + \gamma V_{\pi}(s_{t+1}) \mid s_t = s] \\
 &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma Q_{\pi}(s_{t+1}, \pi'(s_{t+1})) \mid s_t = s] \\
 &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid s_t = s] \\
 &= V_{\pi'}(s).
 \end{aligned}$$

也就是说，只要每个状态的一步策略改善成立，长期价值也会改善。但在函数逼近下，优势估计和策略参数化都会带来误差，可能在某些状态出现

$$\sum_a \tilde{\pi}(a|s) A_{\pi}(s, a) < 0.$$

因此实际算法不能只追求替代目标增大，还必须限制策略变化幅度。

8.5 从精确替代目标到可采样目标

为了把精确式变成可优化目标，TRPO/PPO 使用两个近似技巧。

8.5.1 忽略状态分布变化

精确式中状态分布是 ρ_{π} 。为了用旧策略采样到的数据估计目标，先用旧策略状态分布替代新策略状态分布：

$$L_{\pi}(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\pi}(s) \sum_a \tilde{\pi}(a|s) A_{\pi}(s, a).$$

这个 $L_{\pi}(\tilde{\pi})$ 就是替代回报函数。它在 $\tilde{\pi}$ 接近 π 时近似真实回报。

8.5.2 用重要性采样处理动作分布

动作仍然来自新策略 $\tilde{\pi}$ ，但数据由旧策略 $\pi_{\theta_{\text{old}}}$ 采集。令采样分布

$$q(a|s) = \pi_{\theta_{\text{old}}}(a|s),$$

则

$$\sum_a \pi_{\theta}(a|s) A_{\theta_{\text{old}}}(s, a) = \mathbb{E}_{a \sim q} \left[\frac{\pi_{\theta}(a|s)}{q(a|s)} A_{\theta_{\text{old}}}(s, a) \right].$$

代入 $q = \pi_{\theta_{\text{old}}}$ ，得到常用替代目标：

$$L_{\theta_{\text{old}}}(\theta) = \eta(\pi_{\theta_{\text{old}}}) + \mathbb{E}_{s \sim \rho_{\theta_{\text{old}}}, a \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_{\theta_{\text{old}}}(s, a) \right].$$

其中的概率比记为

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

概率比大于 1 表示新策略比旧策略更倾向于选择该动作，小于 1 则相反。

8.6 替代目标与真实目标的一阶一致性

替代目标不是随意构造的。它在旧参数 θ_{old} 处与真实回报有相同函数值和相同一阶梯度：

$$L_{\theta_{\text{old}}}(\theta_{\text{old}}) = \eta(\pi_{\theta_{\text{old}}}),$$

$$\nabla_{\theta} L_{\theta_{\text{old}}}(\theta)|_{\theta=\theta_{\text{old}}} = \nabla_{\theta} \eta(\pi_{\theta})|_{\theta=\theta_{\text{old}}}.$$

因此，在 θ_{old} 附近，能改善 L 的策略通常也能改善原回报 η 。关键问题就变成：到底可以走多远？

8.7 单调改进下界

课件给出如下性能下界。令

$$\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}), \quad \epsilon = \max_{s,a} |A_{\pi}(s, a)|,$$

则

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon}{(1-\gamma)^2} \alpha^2.$$

该不等式说明：真实回报至少等于替代目标减去一个与策略距离平方有关的惩罚项。若新旧策略距离很小，替代目标就是可靠的；若策略跳得太远，替代目标可能虚高。

利用 KL 散度约束可以得到更实用的保守目标。课件写成

$$\eta(\tilde{\pi}) \geq L_{\pi}(\tilde{\pi}) - CD_{\text{KL}}^{\max}(\pi, \tilde{\pi}), \quad C = \frac{2\epsilon\gamma}{(1-\gamma)^2}.$$

于是定义

$$M_i(\pi) = L_{\pi_i}(\pi) - CD_{\text{KL}}^{\max}(\pi_i, \pi).$$

由于

$$\eta(\pi_{i+1}) \geq M_i(\pi_{i+1}), \quad \eta(\pi_i) = M_i(\pi_i),$$

所以

$$\eta(\pi_{i+1}) - \eta(\pi_i) \geq M_i(\pi_{i+1}) - M_i(\pi_i).$$

因此，如果每次都最大化 M_i ，即让

$$M_i(\pi_{i+1}) \geq M_i(\pi_i),$$

就能保证策略性能非递减。这就是 TRPO “信赖域”思想的理论来源：**提高替代目标，同时用策略距离惩罚或约束避免走出可信区域。**

8.8 TRPO：带 KL 约束的策略优化

TRPO (Trust Region Policy Optimization) 把上面的保守目标转化为带约束优化问题。直接使用惩罚因子 C 往往会使步长过小，所以 TRPO 采用约束形式：

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}_{s \sim \rho_{\theta_{\text{old}}}, a \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A_{\theta_{\text{old}}}(s, a) \right], \\ \text{s.t.} \quad & \mathbb{E}_{s \sim \pi_{\theta_{\text{old}}}} [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s))] \leq \delta. \end{aligned}$$

这里有两个重要实用技巧。

- **最大 KL 改为平均 KL**：原理论中 $D_{\text{KL}}^{\max}$ 要对所有状态约束，实际状态空间可能无穷大，因此改用旧策略状态分布下的平均 KL。
- **旧策略状态分布采样**：用当前旧策略 $\pi_{\theta_{\text{old}}}$ 运行得到样本，再用样本均值近似期望。

于是样本版本可以写成

$$\begin{aligned} \max_{\theta} \quad & \frac{1}{N} \sum_{t=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t, \\ \text{s.t.} \quad & \frac{1}{N} \sum_{t=1}^N D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t)) \leq \delta. \end{aligned}$$

8.9 TRPO 的近似求解：共轭梯度与线性搜索

TRPO 的约束优化仍然复杂。课件采用局部近似：把目标函数在 θ_{old} 处线性化，把 KL 约束二次近似。令

$$g = \nabla_{\theta} L_{\theta_{\text{old}}}(\theta)|_{\theta=\theta_{\text{old}}}, \quad d = \theta - \theta_{\text{old}}.$$

平均 KL 的 Hessian 记为 Fisher 信息矩阵

$$A_{ij} = \frac{\partial}{\partial \theta_i} \frac{\partial}{\partial \theta_j} \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \Big|_{\theta=\theta_{\text{old}}}.$$

则近似优化问题为

$$\max_d g^T d, \quad \text{s.t.} \quad \frac{1}{2} d^T A d \leq \delta.$$

其无约束方向满足

$$A d = g.$$

由于深度网络参数很多，不显式求逆 A^{-1} ，而是用共轭梯度法近似求解线性方程 $A d = g$ 。若得到搜索方向 d ，再用约束确定初始步长。把 βd 代入二次约束：

$$\delta \approx \frac{1}{2} (\beta d)^T A (\beta d) = \frac{1}{2} \beta^2 d^T A d,$$

所以

$$\beta = \sqrt{\frac{2\delta}{d^T A d}}.$$

候选更新为

$$\theta_{\text{new}} = \theta_{\text{old}} + \beta d.$$

实际算法还会进行线性搜索：如果候选更新不满足 KL 约束，或替代目标没有改善，就缩小 β ，直到满足约束并带来目标提升。

TRPO 的基本流程为：

1. 用当前策略运行 T 步或采集 N 条轨迹；
2. 用所有时间步数据估计优势函数 $\hat{A}_t$ ；
3. 形成替代目标和平均 KL 约束的采样估计；
4. 用共轭梯度法求近似自然梯度方向；
5. 用线性搜索选择满足 KL 约束的步长；
6. 更新策略参数。

8.10 PPO：用裁剪目标近似信赖域

TRPO 理论清晰，但实现上需要二阶近似、Fisher 矩阵向量积、共轭梯度和线性搜索。PPO (Proximal Policy Optimization) 保留“限制新旧策略差异”的思想，但把复杂约束换成一阶优化可直接处理的裁剪目标。

普通策略梯度目标可写为

$$L^{PG}(\theta) = \mathbb{E}_t \left[\log \pi_\theta(a_t | s_t) \hat{A}_t \right].$$

TRPO 的替代目标核心是

$$\mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right], \quad r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

PPO 的 clipped 目标为

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right].$$

其中 ε 控制允许的概率比变化范围。

这个目标可以分两种情况理解。

- 当 $\hat{A}_t > 0$ 时, 该动作比平均好, 优化会倾向于增大它的概率。但若 $r_t(\theta) > 1 + \varepsilon$, 裁剪项不再继续增大, 防止新策略过度偏向该动作。
- 当 $\hat{A}_t < 0$ 时, 该动作比平均差, 优化会倾向于减小它的概率。但若 $r_t(\theta) < 1 - \varepsilon$, 裁剪项限制继续下降, 防止一步把动作概率压得过低。

由于取了两个目标中的较小者, PPO 实际上在优化一个更保守的下界, 效果类似于不用显式 KL 约束也能限制策略更新幅度。

8.11 PPO 的完整目标与 GAE

实际 PPO 常采用 Actor-Critic 结构, 同时学习策略和值函数。课件给出的完整目标为

$$L_t^{\text{CLIP}+VF+S}(\theta) = \hat{\mathbb{E}}_t [L_t^{\text{CLIP}}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)].$$

其中 L_t^{VF} 是值函数损失, $S[\pi_\theta](s_t)$ 是策略熵, c_1, c_2 是权重。熵项鼓励策略保持一定随机性, 避免过早失去探索。

为了减小优势估计的方差, PPO 通常使用广义优势估计 (Generalized Advantage Estimation, GAE):

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1},$$

其中

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

λ 控制偏差和方差的折中: λ 越小, 估计越接近 TD, 方差较小但偏差较大; λ 越大, 估计越接近 Monte Carlo, 偏差较小但方差较大。

PPO 的 Actor-Critic 式伪代码可以概括为:

1. 对每轮迭代, 使用旧策略 $\pi_{\theta_{\text{old}}}$ 与环境交互, 多个 actor 各运行 T 个时间步;
2. 计算每个时间步的优势估计 $\hat{A}_1, \dots, \hat{A}_T$;
3. 用 K 个 epoch 和小批量数据优化替代目标 $L^{\text{CLIP}+VF+S}$;
4. 令 $\theta_{\text{old}} \leftarrow \theta$, 进入下一轮采样。

8.12 TRPO 用到的信息论概念

TRPO 和 PPO 都需要衡量策略分布的变化。常用的信息论量包括熵、交叉熵和 KL 散度。

离散分布 p 的熵为

$$H(p) = - \sum_x p(x) \log p(x),$$

连续分布的微分熵形式为

$$H(p) = - \int p(x) \log p(x) dx.$$

熵越大, 分布越不确定。PPO 目标中的熵奖励就是利用这一点鼓励探索。

交叉熵衡量用分布 q 的编码方式描述来自分布 p 的样本时的平均编码长度:

$$H(p, q) = - \sum_x p(x) \log q(x).$$

KL 散度定义为

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p).$$

它衡量两个概率分布之间的差异。TRPO 中用

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s))$$

约束每个状态下新旧策略动作分布的变化。

8.13 TRPO 与 PPO 对比

- **共同点：**二者都从替代目标出发，都使用旧策略采样数据和概率比 $r_t(\theta)$ ，目标都是避免策略更新过大导致性能崩溃。
- **TRPO：**使用平均 KL 散度显式约束信赖域，理论上与单调改进下界联系紧密，但需要共轭梯度和线性搜索，实现复杂。
- **PPO：**用 clip 目标近似限制概率比变化，可用普通随机梯度方法多轮小批量优化，实现简单，稳定性好，因此实际应用更广。

可以把 TRPO 理解为“严格求解带 KL 约束的保守策略改进”，把 PPO 理解为“用裁剪目标实现近似的保守策略改进”。二者的核心问题相同：**在提高回报和控制策略变化之间取得平衡。**

第九章 确定性策略梯度方法

9.1 从随机策略梯度到确定性策略梯度

随机策略在状态 s 下输出动作分布：

$$a \sim \pi_{\theta}(a|s).$$

确定性策略则直接输出动作：

$$a = \mu_{\theta}(s).$$

随机策略梯度需要对状态和动作都积分：

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a)].$$

而确定性策略梯度只需要对状态分布取期望，不需要对动作积分，因此在高维连续动作空间中更高效。

9.2 Actor-Critic 框架

如果直接用采样回报估计策略梯度，会产生较大方差。

Actor-Critic 使用值函数逼近器作为 Critic：

- **Actor**：策略网络，更新 π_{θ} 或 μ_{θ} ；
- **Critic**：值函数网络，估计 $Q(s, a)$ 并指导 Actor 更新。

在随机策略梯度中，若用函数逼近 $Q^w(s, a)$ 替代真实 $Q^{\pi}(s, a)$ ，可能引入偏差。满足特定条件的函数逼近器称为**相容函数逼近器**。

9.3 确定性值函数

对于确定性策略 μ ，动作值函数满足：

$$Q^{\mu}(s_t, a_t) = \mathbb{E}[r(s_t, a_t) + \gamma Q^{\mu}(s_{t+1}, \mu(s_{t+1}))].$$

由于下一动作由 $\mu(s_{t+1})$ 确定，期望只来自环境转移和回报随机性。

9.4 确定性策略梯度定理

确定性策略梯度定理给出：

$$\nabla_{\theta} J(\mu_{\theta}) = \mathbb{E}_{s \sim \rho^{\mu}} \left[\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q^{\mu}(s, a) \Big|_{a=\mu_{\theta}(s)} \right].$$

其含义是：先看策略参数如何改变动作，再看动作变化会如何改变动作值函数。这正是链式法则在强化学习中的体现。

9.5 确定性 Actor-Critic

确定性 Actor-Critic 通常是 off-policy 的：

- 行为策略用于探索，可以是随机策略；
- 目标策略是确定性策略 $\mu_{\theta}(s)$ ；
- Critic 估计 $Q^{\mu}(s, a)$ ；
- Actor 沿确定性策略梯度方向更新。

Critic 的 TD 目标通常为：

$$y = r + \gamma Q(s', \mu(s'); w).$$

9.6 DDPG

DDPG (Deep Deterministic Policy Gradient) 将确定性策略梯度与深度神经网络结合，适用于连续动作控制问题。

DDPG 继承了 DQN 的两个重要技巧：

- **经验回放**：缓解样本相关性，提高数据利用率。
- **目标网络**：分别为 Actor 和 Critic 维护目标网络，提高训练稳定性。

Critic 目标为：

$$y = r + \gamma Q'(s', \mu'(s'); w'),$$

Critic 最小化：

$$L(w) = \mathbb{E} \left[(y - Q(s, a; w))^2 \right].$$

Actor 使用梯度：

$$\nabla_{\theta} J \approx \mathbb{E} \left[\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q(s, a; w) \Big|_{a=\mu_{\theta}(s)} \right].$$

训练时为了探索，常在确定性动作上加入噪声：

$$a = \mu_{\theta}(s) + \mathcal{N}.$$

9.7 TD3

DDPG 存在值函数过估计、方差较大以及 Actor-Critic 耦合过强等问题。TD3 (Twin Delayed DDPG) 提出三点改进:

- **双 Critic:** 使用两个 Critic, 目标值取较小者, 缓解过估计。
- **目标策略平滑:** 在目标动作上加噪声, 使 Critic 对动作小扰动更平滑。
- **延迟策略更新:** Critic 更新多次后再更新 Actor 和目标网络, 降低耦合带来的不稳定。

9.8 D4PG

D4PG (Distributed Distributional Deep Deterministic Policy Gradient) 在 DDPG 基础上进一步改进 Critic 学习。它使用:

- 分布式价值函数;
- 多个并行 Actor;
- N-step 回报;
- 优先经验回放。

其核心思想是: Actor 的更新质量依赖 Critic, 因此改进 Critic 学习通常会直接提升策略更新质量。

第十章 多智能体强化学习

10.1 多智能体强化学习的分类

多智能体强化学习（Multi-Agent Reinforcement Learning, MARL）研究多个智能体在同一环境中同时学习和决策的问题。

按照任务关系，可以分为三类：

- **完全协作**：所有智能体目标一致，共同最大化团队回报。
- **完全竞争**：智能体目标完全相反，例如双人零和博弈。
- **混合任务**：智能体之间既可能合作，也可能竞争。

10.2 多智能体强化学习的挑战

将单智能体强化学习直接用于多智能体系统会遇到困难：

- **环境非静态**：单智能体 MDP 假设环境动态固定；但从某个智能体视角看，其他智能体也在学习和改变策略，因此环境是动态变化的。
- **最优策略概念改变**：在竞争或混合任务中，各智能体目标可能冲突，不能简单求一个使所有人都最优的策略。
- **联合动作空间巨大**：多个智能体的联合动作空间随智能体数量指数增长。

因此，多智能体强化学习需要结合博弈论和强化学习。

10.3 博弈论基本概念

博弈可以按不同方式分类：

- **静态博弈**：参与者同时选择动作，常用矩阵博弈表示。
- **动态博弈**：参与者动作有先后顺序，常用博弈树表示。
- **完美信息博弈**：参与者知道相关信息，例如棋类游戏。
- **不完美信息博弈**：参与者信息不完全，例如牌类游戏。

完美信息静态博弈包括：

- **参与者**：参与博弈的智能体；

- **动作空间**：每个智能体可选动作集合；
- **联合动作**：所有智能体动作组成的有序集合；
- **策略**：智能体在动作空间上的概率分布；
- **回报函数**：每个智能体在联合动作下得到的收益。

策略可以分为：

- **纯策略**：某个动作选择概率为 1；
- **混合策略**：在多个动作上分配概率。

10.4 Best Response 与纳什均衡

在博弈中，一个玩家的收益依赖其他玩家的策略。因此不能只考虑单个智能体独立最大化收益。

Best response 指在其他玩家策略固定时，某玩家的最优策略。若所有玩家的策略都是对其他玩家策略的 best response，则构成纳什均衡（Nash equilibrium）。

纳什均衡的含义是：

- 每个玩家都是理性的；
- 没有玩家能通过单独改变自己的策略来提高回报。

例如囚徒困境中，“坦白，坦白”是纳什均衡，因为任一囚徒单独改变策略都不能改善自己的结果。

10.5 零和博弈与 Minimax

在双人零和博弈中，一个玩家的收益等于另一个玩家的损失。此时常用 minimax 思想求解均衡。对于玩家 k ，若对手动作为 a' ，其值可写成：

$$V_k(s) = \min_{a'} \max_{\sigma \in \Delta(A)} \sum_{a \in A} \sigma(a) Q_k(s, a, a').$$

这表示：在最坏对手动作下，选择使自己收益最大的策略。双人零和矩阵博弈的纳什均衡可转化为线性规划求解。

10.6 马尔科夫博弈

现实多智能体问题通常涉及状态转移，因此需要用马尔科夫博弈（stochastic game）描述。

马尔科夫博弈包括：

- 智能体数量 n ；
- 状态空间 S ；
- 每个智能体的动作空间 A_i ；
- 联合动作 $\mathbf{a} = (a_1, \dots, a_n)$ ；

- 状态转移概率 $P(s'|s, \mathbf{a})$;
- 每个智能体的回报函数 $r_i(s, \mathbf{a})$;
- 每个智能体的局部策略 $\pi_i(a_i|s)$ 。

智能体 i 的值函数为:

$$V_i^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_i(t+1) \middle| s(0) = s \right],$$

其中联合策略为:

$$\pi = (\pi_1, \dots, \pi_n).$$

多智能体值函数不仅依赖自己的策略, 也依赖其他智能体的策略。

10.7 多智能体 Q-learning

多智能体中, 动作值函数通常依赖联合动作:

$$Q_i(s, \mathbf{a}).$$

更新形式为:

$$Q_i(s, \mathbf{a}) \leftarrow Q_i(s, \mathbf{a}) + \alpha [r_i + \gamma V_i(s') - Q_i(s, \mathbf{a})].$$

关键问题是如何得到下一状态价值 $V_i(s')$ 。常见方法包括:

- **对手建模**: 估计其他智能体联合动作频率, 再计算期望价值。
- **Minimax-Q**: 在完全竞争零和博弈中, 用 minimax 原则计算下一状态价值。
- **Nash-Q**: 假设下一个状态处各智能体执行纳什均衡策略。
- **相关均衡或 Stackelberg 均衡**: 用其他均衡概念替代纳什均衡。

10.8 典型多智能体算法

- **完全协作强化学习**: 所有智能体共享团队目标, 常用于无人机覆盖、机器人协作等任务。
- **Minimax-Q**: 用于双人零和完全竞争任务, 通过 minimax 计算值函数。
- **Nash-Q**: 用纳什均衡更新行为值函数, 适用于一般和随机博弈, 但收敛条件较严格。
- **朋友或敌人 Q-learning**: 将其他智能体划分为朋友或敌人, 朋友共同最大化回报, 敌人共同最小化回报。
- **WoLF-PHC**: “Win or Learn Fast”, 表现差时快速学习, 表现好时谨慎学习, 并结合策略爬山更新策略。

10.9 深度多智能体强化学习

深度强化学习可以用于多智能体连续控制任务。典型方法是 MADDPG。

MADDPG (Multi-Agent DDPG) 的思想是：

- 每个智能体有自己的 Actor，只根据局部观测选择动作；
- 训练时 Critic 可以使用全局信息，例如所有智能体的状态和动作；
- 执行时 Actor 只依赖本智能体观测，实现集中训练、分散执行。

这种框架可以缓解多智能体环境的非静态性，是深度多智能体强化学习中的重要思想。

10.10 多智能体强化学习小结

多智能体强化学习与单智能体强化学习的根本区别在于：其他智能体的策略会影响环境动态和自己的回报。

本章重点可以概括为：

- 多智能体任务分为协作、竞争和混合；
- 单智能体 MDP 在多智能体中变成马尔科夫博弈；
- 最优策略概念通常被均衡概念替代；
- 多智能体 Q-learning 需要考虑联合动作和其他智能体策略；
- 深度 MARL 常采用集中训练、分散执行的框架。